NIPT 检测时点的分层优化与女胎异常判定

摘要

针对 NIPT 检测时点与胎儿异常判定问题，本文首先把“孕妇”“抽血事件”和“测序记录”区分为三个层级，避免将重复检测误作独立样本。附件共含男胎 1082 条、267 名孕妇和 1063 个技术事件，女胎 605 条、147 名孕妇，其中 AB 列阳性 67 条。全部验证与重采样均以孕妇代码为隔离单元。

问题一对事件级 Y 染色体浓度作 logit 变换，以孕周限制性样条、BMI 和预先指定质量指标建立随机截距混合模型，并用交换相关结构的聚类稳健 GEE 复核。孕周整体检验显著（$p = 1.09e-42$），BMI 效应方向为负（$p = 0.044$）；固定效应边际 $R^2 = 0.156$，含母体差异的条件 $R^2 = 0.771$，组内相关系数为 0.729。

问题二将 $Y \geq 4\%$ 定义为达标，利用二项 GEE 估计孕周–BMI 条件达标概率，再把个体首次达到 90% 可靠性所需孕周按 BMI 排序，以有序动态规划压缩为 4 个连续区间。所得政策的平均额外等待为 0.981 周；技术重复估计的单次测量标准差为 0.0047，200 次测量误差模拟和 200 次孕妇自助法均无失败。最高 BMI 组到 25 周仍有少数个体未达到目标，故 25 周只作为题设检测上限而非可靠性保证。

问题三只使用检测前可获得的母体信息，比较 BMI 基线、弹性网逻辑回归和单调梯度提升，并实施按孕妇分组的 5×4 嵌套验证与概率校准。最终选择单调梯度提升，组外 ROC-AUC、PR-AUC 和 Brier 分数分别为 0.673、0.922 和 0.109；个体时点再压缩为 2 个 BMI 连续组。但多因素个体差异不能被 BMI 顺序充分表达，压缩平均后悔达到 6.249 周，明显高于问题二的 0.981 周，因此不宣称该统一分组优于问题二。

问题四严格以 AB 非空为监督标签，比较预设三西格玛 Z 值规则、类权重弹性网和树模型，调参与阈值均在内层孕妇分组数据完成。最终选择弹性网，其组外 PR-AUC 为 0.411、ROC-AUC 为 0.738，灵敏度和特异度分别为 0.642、0.725。结果只反映对附件 AB 检测标签的复现能力；女胎 AE 全为“是”，不能将上述性能解释为出生结局诊断性能。

关键词：随机截距回归分析　分组动态规划　嵌套交叉验证与误差分析　弹性网逻辑回归

一、 问题重述

NIPT 通过母体外周血中的胎儿游离 DNA 判断染色体异常。题面指出，孕 10–25 周可检测胎儿性染色体浓度；男胎 Y 染色体浓度达到或超过 4% 时，结果基本满足准确性要求。发现异常越晚，治疗窗口越短，潜在风险越高。附件数据来自高 BMI 人群较多的地区，并包含同一孕妇多次采血、同次采血重复测序和测序失败等复杂结构 [1]。

本文需要完成四项任务：

1. 建立 Y 染色体浓度与孕周、BMI 及质量指标的关系模型，并检验显著性；
2. 仅依据 BMI 形成连续分组，为每组给出使失败与延迟风险折中的 NIPT 时点，并评价检测误差；
3. 综合身高、体重、年龄和孕产史等因素，验证多因素模型是否真正改善未见孕妇的达标预测，再输出可执行的 BMI 分组时点；
4. 以女胎 AB 列非整倍体检测结果为标签，综合 Z 值、GC、读段质量和 BMI 等因素给出异常判定方法。

题目中的“最佳时点”不是普适临床指南，而是给定附件分布、4% 阈值和本文可靠性目标下的决策建议；问题四的目标也不是出生后确诊结局，而是 AB 检测标签。

二、 问题分析

2.1 相关结构是四问共同的首要约束

一行数据并不等于一名独立孕妇。若直接随机拆分记录，同一孕妇的固定体征和历史检测会同时出现在训练集与验证集，预测指标将明显偏乐观。为此，本文定义技术事件

$$e = (\text{孕妇代码}, \text{检测日期}, \text{抽血次数}),$$

问题一至三先把同事件男胎连续测量取中位数，问题四保留每条女胎检测记录，但所有外层验证都按孕妇整体分组。

2.2 问题一：关系刻画而非因果推断

Y 浓度取值有界且母体内多次观测相关。孕周关系可能非线性，因此主模型应同时处理边界、样条曲线和随机母体差异；BMI 与孕周交互只作为候选项，以嵌套模型比较决定是否保留。观察性检测安排可能受既往结果影响，故结论限于条件相关。

2.3 问题二：先形成个体可靠时点，再做可执行压缩

题面没有给出精确临床效用常数。本文将早检失败风险转化为“达到给定可靠性水平”的约束，先求每名孕妇最早可靠时点，再以连续 BMI 分组的共同周数不早于组内个体需求为约束，最小化额外等待与组数复杂度。这样既避免主观拼接经验区间，也能明确解释分组损失。

2.4 问题三：多因素增益必须来自未见孕妇

年龄、身高、体重、孕产史和受孕方式可能提供 BMI 之外的信息，但测序后的 GC、读段等变量在确定首次检测时点之前不可得，若纳入会造成时间信息泄漏。因此只使用检测前信息，按孕妇嵌套验证候选模型，并把个体策略再次压缩为 BMI 连续区间。

2.5 问题四：标签不平衡且不是确诊金标准

女胎 AB 阳性率仅为 0.111，且 2 个同次技术事件、43 名重复检测孕妇存在标签变化。固定 Z 值规则作为透明基线；弹性网用于稳定处理相关特征，树模型检查非线性增益。评价以 PR-AUC 为主，同时给出 ROC-AUC、灵敏度、特异度、平衡准确率、Brier 分数、校准和孕妇自助区间。

三、 模型假设

假设 1：同一孕妇的多次记录相关。解释：孕妇代码作为随机效应、GEE 聚类、交叉验证和自助法的最小单元；任何分折中训练、验证孕妇集合均不相交。

假设 2：同孕妇、同日期、同抽血次数的重复记录为技术重复。解释：男胎连续变量在事件内取中位数，重复差异仅用于估计测量误差；女胎标签不一致事件在敏感性分析中单列。

假设 3：题面 $Y \geq 4\%$ 是男胎 NIPT 基本可靠的操作性阈值。解释：问题二、三据此定义二元达标变量，同时对 3.5%、4.5% 和不同可靠性目标作情景复核。

假设 4：具体风险权重不是固定临床常数。解释：题面只给出早、中、晚期风险的定性递增关系，故本文把结果限定为给定可靠性和复杂度惩罚下的最优政策，不宣称唯一临床最优。

假设 5：问题四的监督目标是 AB 非空。解释：AE 出生后健康列在女胎中没有变异，不能代替 AB，也不能作为外部确诊金标准。

假设 6：缺乏随机干预，关系模型不作因果解释。解释：孕周覆盖与复检安排可能存在选择偏差，外推到低 BMI 或不同地区人群需重新验证。

四、 符号说明

表 1: 主要符号

符号	含义
i, j	第 i 名孕妇及其第 j 个技术事件
t_{ij}	检测孕周
b_i	BMI
Y_{ij}	男胎 Y 染色体浓度
A_{ij}	达标指示量 $\mathbf{1}(Y_{ij} \geq 0.04)$
$p(t, b, \boldsymbol{x})$	条件达标概率
ρ	策略可靠性目标，主分析取 0.90
w_i	第 i 名孕妇的最早可靠检测孕周
G_k	第 k 个连续 BMI 分组
τ	女胎异常分类阈值，只在内层训练数据选择

五、 数据预处理

5.1 原始文件与字段规范化

正式运行先核对题面 PDF 与 Excel 的 SHA256，所有派生文件写入独立冻结目录，原件不改写。孕周字符串同时兼容 “20w” 和 “20w+1”；混合的整数日期与 Excel 日期对象统一为无时区日期；女胎 1 个 BMI 缺失值由同一行体重与身高按

$$\mathrm{BMI} = \frac{\text{体重/kg}}{(\text{身高/m})^2}$$

确定性重算，并保留重算标记。女胎空白的 Y 染色体列作为结构性缺失，不作特征。

表 2: 附件数据结构与技术重复审计

数据集	记录数	孕妇数	技术事件	重复事件	AB 阳性行
男胎	1082	267	1063	19	126
女胎	605	147	590	15	67

5.2 技术重复与纵向事件

男胎 1082 条记录被聚合为 1063 个事件，对连续量取事件内中位数。19 对男胎同次抽血重复测序用于估计 Y 浓度技术误差，而不增加独立样本量。女胎保留 605 条判定记录，但后续分折始终按 147 名孕妇整体隔离；对标签不一致技术事件另做排除敏感性。

5.3 建模变量与泄漏控制

问题一使用事件级 Y 浓度、孕周、BMI、年龄、身高、受孕方式和测序质量指标。问题二仅用孕周与 BMI。问题三只纳入检测时已知的孕周、BMI、年龄、身高、体重、受孕方式、怀孕次数和生产次数，明确排除检测后才产生的 GC 与读段信息。问题四使用 13/18/21/X 染色体 Z 值、相应 GC、总 GC、读段数及比例、BMI、孕周和基本体征；AB 原始文字、解析标签和 AE 均从特征中排除。

六、 模型建立与求解

6.1 问题一：Y 浓度纵向关系模型

为稳定有界响应，令

$$Z_{ij} = \log \frac{Y_{ij} + \varepsilon}{1 - Y_{ij} + \varepsilon}, \qquad \varepsilon = 10^{-6}.$$

设 $B_m(t)$ 为四维三次样条基函数，主模型为

$$Z_{ij} = \beta_0 + \sum_{m=1}^{4} \beta_m B_m(t_{ij}) + \beta_b \widetilde{b}_{ij} + \boldsymbol{\gamma}^{\mathsf{T}} \widetilde{\boldsymbol{x}}_{ij} + u_i + \epsilon_{ij},$$

其中 $u_i \sim N(0, \sigma_u^2)$ 为母体随机截距，$\widetilde{\boldsymbol{x}}$ 包含年龄、身高、受孕方式、GC、比对率、重复率和过滤率。另拟合线性孕周模型及样条–BMI 交互候选，以 AIC、似然比块检验和简约性选型。

表 3: 问题一纵向候选模型比较

模型	收敛	AIC	BIC	固定效应参数数
linear_baseline	True	848.088	907.714	10
interaction_candidate	True	829.097	923.505	17
selected	True	823.89	898.423	13
without_gestational_week	True	1129.321	1183.978	9
without_bmi	True	829.43	898.994	12
without_quality	True	824.109	878.767	9

所选模型固定效应边际 $R^2 = 0.156$，条件 $R^2 = 0.771$，ICC 为 0.729，说明相当部分变异来自稳定母体差异。交换相关 GEE[2] 的稳健检验显示孕周整体 $p = 1.09e - 42$，BMI 系数方向为负且 $p = 0.044$。图 1 展示不同 BMI 分位下的预测关系。

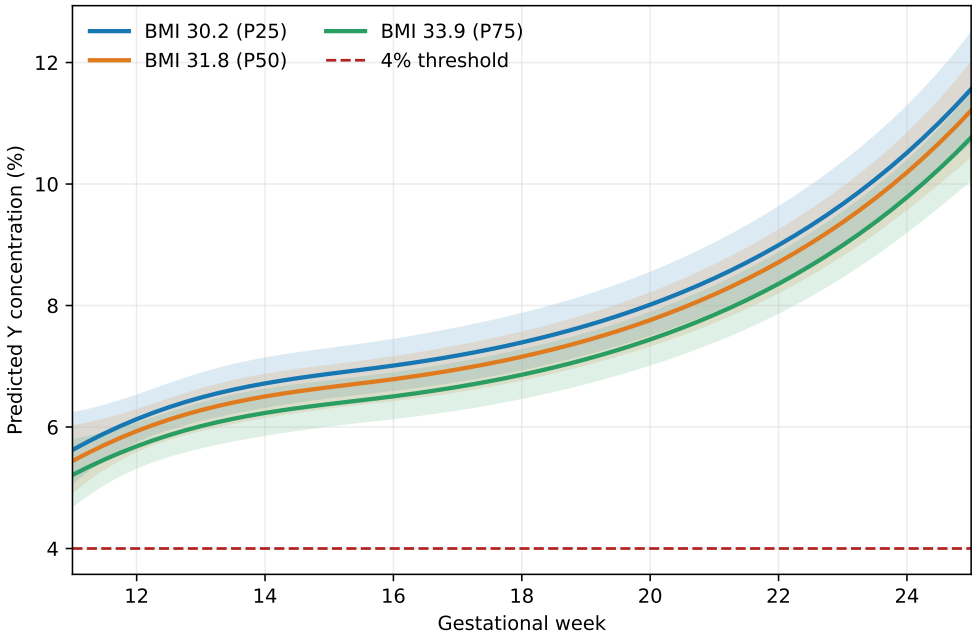

图 1: 问题一不同 BMI 水平下的 Y 浓度预测曲线及置信带

6.2 问题二：BMI 分组与检测时点

定义 $A_{ij} = \mathbf{1}(Y_{ij} \geq 0.04)$，以母体为聚类单元拟合二项 GEE：

$$\operatorname{logit}\{p(t,b)\} = \alpha_0 + \alpha_t(t-15) + \alpha_b\widetilde{b}.$$

对每名孕妇取其代表 BMI，最早可靠时点为

$$w_i = \min\{t \in [10,25] : p(t,b_i) \geq \rho\}, \qquad \rho = 0.90.$$

若至 25 周仍未达到，则记录“未达到”并将推荐值截于 25 周，而不宣称可靠性已满足。

将孕妇按 BMI 升序排列。任一连续片段 $[s,e]$ 的共同检测周必须不早于组内最晚个体需求，故片段代价为

$$C(s,e) = \sum_{i=s}^{e}\{\max_{s\leq r\leq e} w_r - w_i\}.$$

令 $D(k,e)$ 表示前 e 人分为 k 组的最小总代价，则

$$D(k,e) = \min_{s}\{D(k-1,s) + C(s+1,e)\},$$

并要求每组不少于 30 名孕妇。最终以

$$J_K = \frac{D(K,n)}{n} + 0.2(K-1)$$

在 $K = 2,\ldots,6$ 中选择组数。主结果如表 4。

表 4: 问题二 BMI 连续分组与推荐检测时点

组别	BMI 下界	BMI 上界	孕妇数	推荐孕周	平均额外等待/周
1	20.7	30.57	110	17.5	0.703
2	30.57	32.74	69	18.8	0.577
3	32.74	35.22	58	20.3	0.824
4	35.22	46.88	30	25	3.233

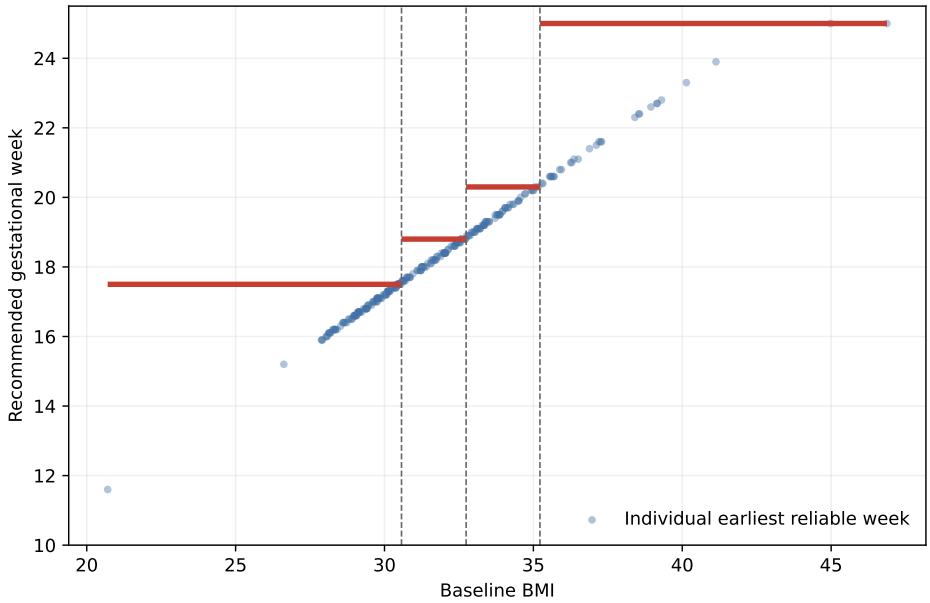

图 2: 问题二连续 BMI 分组及共同推荐孕周

6.3 问题三：多因素达标预测与策略压缩

候选模型包括：只含孕周和 BMI 的逻辑基线；带弹性网惩罚的逻辑回归 [3]；对孕周施加单调递增、对 BMI 施加单调递减约束的梯度提升树 [4]。外层 5 折评估未见孕妇，内层 4 折选择超参数并作 Platt 校准，三类模型在完全相同的外层折上比较 ROC-AUC、PR-AUC、Brier、对数损失和校准误差。

表 5: 问题三按孕妇分组的组外预测性能

模型	ROC-AUC	PR-AUC	Brier	ECE
bmi_baseline	0.589	0.894	0.112	0.025
maternal_elastic_net	0.599	0.879	0.113	0.038
maternal_monotone_boosting	0.673	0.922	0.109	0.027

所选模型为 `maternal_monotone_boosting`，其组外 ROC-AUC 为 0.673，PR-AUC

为 0.922，Brier 为 0.109。利用全体数据的内层分组 OOF 概率拟合最终校准模型，逐孕妇搜索最早 $p \geq 0.90$ 的时点，再沿用问题二动态规划压缩为 2 组。平均压缩后悔为 6.249 周，数值上远大于问题二的 0.981 周；原因是同一 BMI 区间内的年龄、体重和孕产史差异打乱了个体可靠时点的 BMI 单调次序。故表 6 是保护异质个体的保守政策，不作为优于问题二的结论。

表 6: 问题三多因素策略压缩后的 BMI 分组

组别	BMI 下界	BMI 上界	孕妇数	推荐孕周	平均后悔/周
1	20.7	29.76	68	23.3	4.668
2	29.76	46.88	199	25	6.789

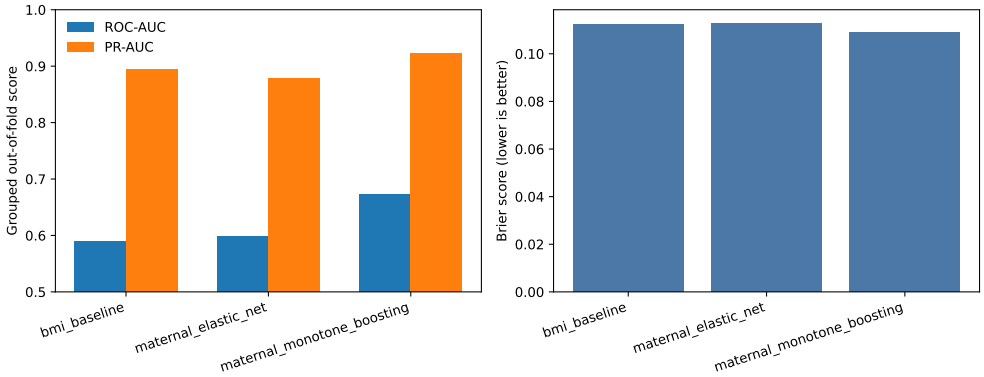

图 3: 问题三按孕妇分组的组外模型比较

6.4 问题四：女胎 AB 异常判定

令 $T_r = \mathbf{1}(\mathrm{AB}_r \text{ 非空})$。预设透明基线为

$$S_r = \max(|Z_{13,r}|, |Z_{18,r}|, |Z_{21,r}|), \qquad \widehat{T}_r = \mathbf{1}(S_r \geq 3),$$

该规则不调参、不因性能差而反向校准。学习模型为类权重弹性网和受控复杂度直方图梯度提升；主评价采用 5 折外层、4 折内层的分层孕妇分组验证。每个外层训练集内，先按 PR-AUC 选超参数，再用内层 OOF 概率校准，并以平衡准确率选阈值 τ。阈值从不接触外层测试数据。

表 7: 问题四嵌套孕妇分组交叉验证性能

模型	PR-AUC	ROC-AUC	灵敏度	特异度	平衡准确率	Brier
z_rule	0.109	0.479	0.06	0.924	0.492	0.145
elastic_net	0.411	0.738	0.642	0.725	0.683	0.082
tree_ensemble	0.207	0.654	0.582	0.641	0.612	0.095

所选模型为 `elastic_net`。其 PR-AUC 为 0.411，ROC-AUC 为 0.738，灵敏度 0.642，特异度 0.725，平衡准确率 0.683，Brier 分数 0.082。固定 Z 规则在本附件上较弱，说明不能仅凭单个通用阈值代替数据质量和母体因素的联合判定。

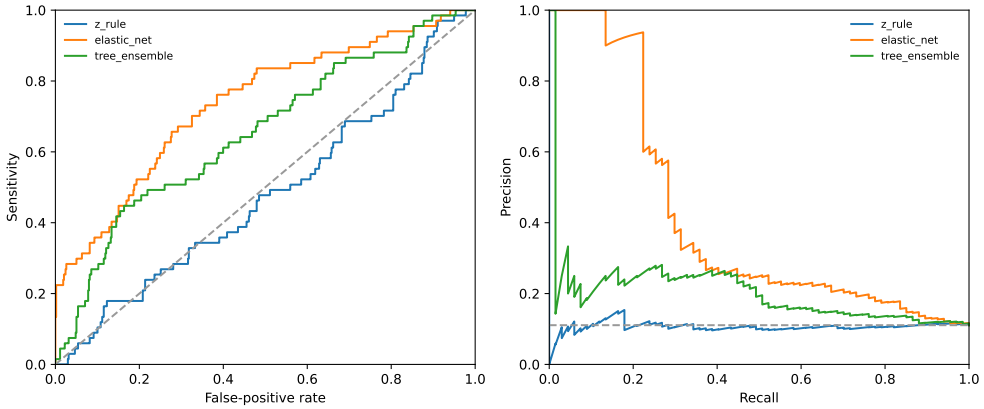

图 4: 问题四三类候选方法的组外 ROC 与 PR 曲线

七、 模型检验与敏感性分析

7.1 问题一诊断与稳健推断

混合模型与线性基线、含交互候选模型均检查收敛、AIC/BIC、标准化残差和影响点。残差尾部偏离正态且存在一定异方差，因此不只依赖混合模型标准误，而用母体聚类的交换相关 GEE 复核关键效应。两种方法下孕周作用稳定、BMI 方向均为负，但 BMI 的统计证据弱于孕周，故不作强因果解释。

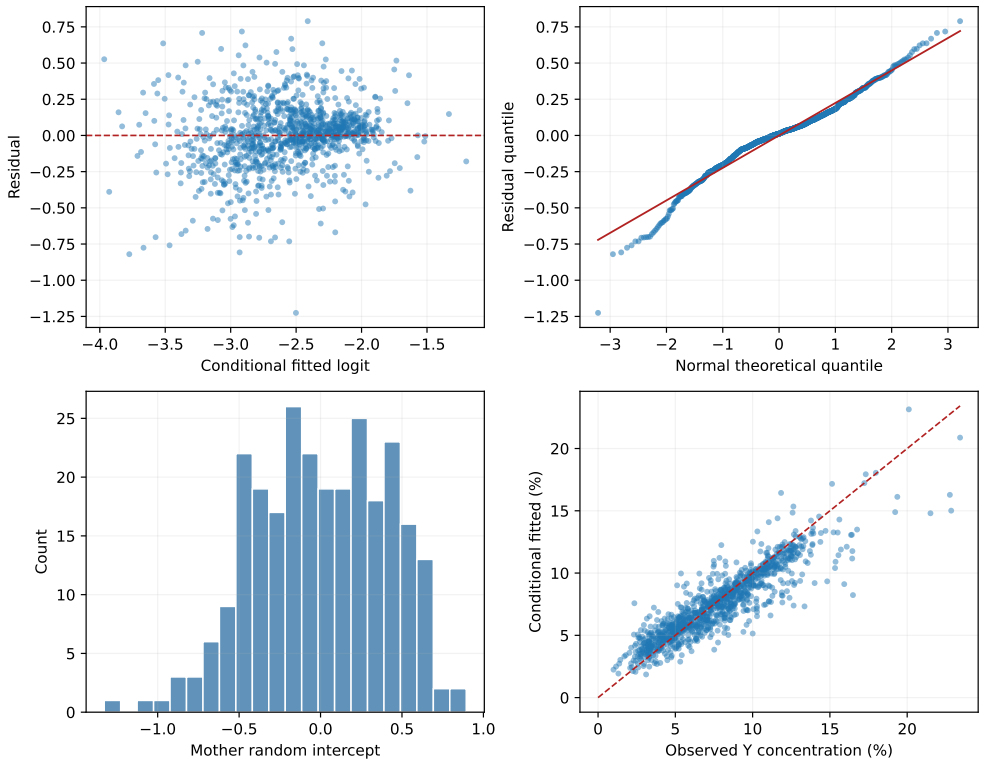

图 5: 问题一混合模型诊断

7.2 问题二检测误差与政策稳定性

19 对同事件技术重复给出 Y 浓度测量标准差 0.0047。主政策在 200 次高斯测量误差模拟和 200 次按孕妇自助法中重新拟合 GEE 与分组，失败次数均为 0。另对 Y 阈值 3.5%/4.0%/4.5% 和可靠性 85%/90%/95% 组合重算。图 6 显示，可靠性要求升高会系统推迟时点，高 BMI 末组最易触及 25 周上限。

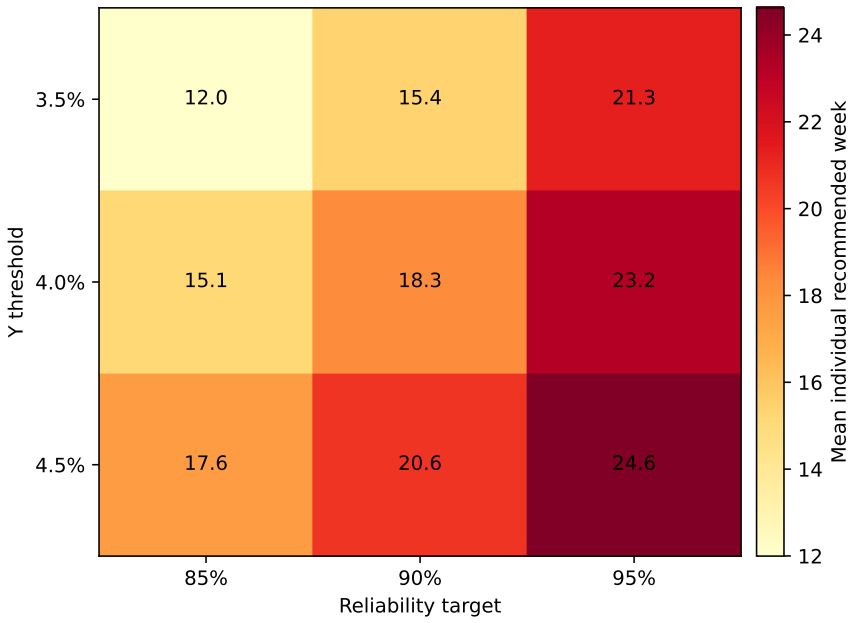

图 6: 问题二阈值与可靠性目标的敏感性

7.3 问题三校准与压缩后悔

五个外层折的训练、验证孕妇交集均为 0，每个事件恰获得一次 OOF 预测。除模型比较外，可靠性目标取 $0.85, 0.90, 0.95$，并将 BMI 测量整体扰动 $-1, 0, +1$ 后重算策略。多因素策略必须同时满足组外概率性能和政策损失不劣于 BMI 基线，若增益有限则只将其解释为预测改进。图 7 与图 8 分别检查概率校准和执行性压缩代价。

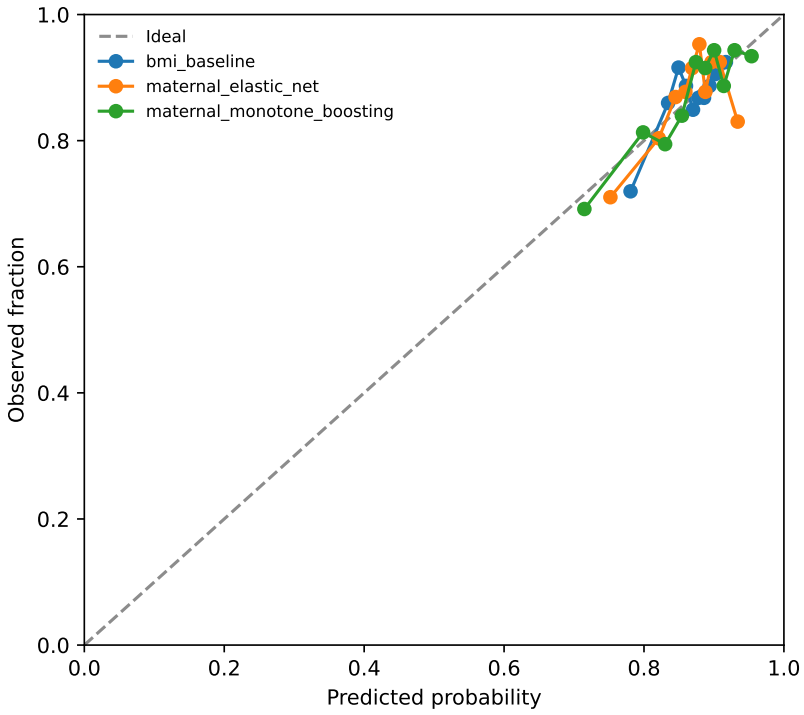

图 7: 问题三候选模型的组外校准

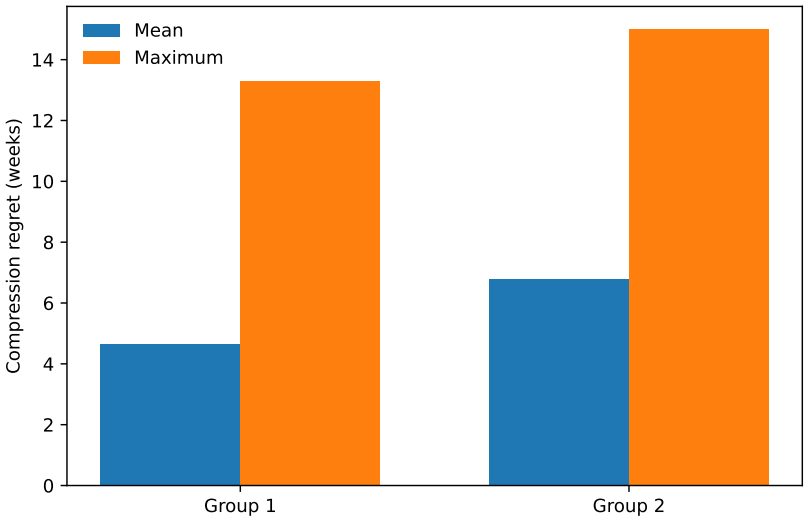

图 8: 问题三 BMI 分组压缩造成的等待后悔

7.4 问题四不平衡、校准与标签噪声

问题四除主指标外，用孕妇为单位进行 500 次自助法，给出 PR-AUC、ROC-AUC、灵敏度、特异度、平衡准确率和 Brier 的不确定区间。标签噪声敏感性包括排除同事件 AB 不一致记录、每事件只留一条记录，并比较 $|Z|$ 阈值 2.5、3.0、3.5。主模型混淆矩阵见表 8，校准与特征效应分别见图 9 和表 9。

表 8: 问题四所选模型的组外混淆矩阵明细

真实标签	预测标签	记录数
0	0	390
0	1	148
1	0	24
1	1	43

表 9: 问题四所选模型的主要特征效应

特征	效应类型	效应值
gc13	标准化对数优势系数	2.54
gc21	标准化对数优势系数	-2.425
age	标准化对数优势系数	-0.306
gestational_week	标准化对数优势系数	0.236
z21	标准化对数优势系数	0.159
x_zscore	标准化对数优势系数	-0.154
weight_kg	标准化对数优势系数	-0.153
bmi	标准化对数优势系数	-0.148

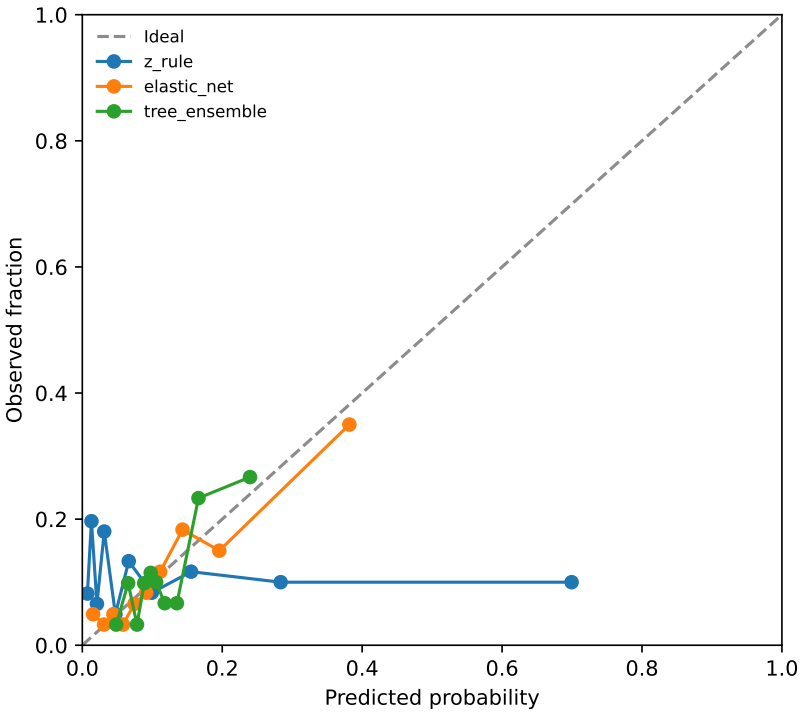

图 9: 问题四候选模型的组外校准

八、 模型评价与推广

8.1 主要优点

第一，数据单位与验证单位一致：技术重复不扩充独立样本，所有重采样和预测评价均按孕妇隔离。第二，关系、预测和决策三类目标分开处理：问题一用纵向统计推断，问题二、三用可复算的可靠时点与动态规划，问题四用嵌套分类验证。第三，透明基线、替代模型、校准、敏感性和母体自助区间共同进入证据链，未收敛拟合或缺失产物会自动阻断正式运行。第四，论文表格和数值由冻结运行 `RUN-C-FORMAL-20260721-001` 自动生成，避免人工转录错误。

8.2 局限性

附件以高 BMI 人群为主，低 BMI 和不同地区的外推范围有限；多次检测时点并非随机安排，关系不能解释为因果。题面只定性说明早、中、晚期风险，本文 90% 可靠性和组数惩罚仍需结合临床偏好选择。问题三多因素模型使用的是同一地区历史数据，即使通过孕妇分组验证，也不能替代前瞻性外部验证。问题四的 AB 是检测标签，2 个技术事件和 43 名孕妇存在标签变化，AE 又无阴性结局，因而模型不能被表述为真实胎儿健康的确诊器。

8.3 推广建议

在新医院或新人群中，应先固定字段映射，按孕妇分层留出独立验证集，重估达标概率与校准，再根据当地可接受的失败率和延迟代价重算 BMI 分组。若能获得产前诊断或出生结局，应将问题四目标替换为独立金标准，并把当前 AB 模型仅作为一个检测流程特征，而非最终标签。

8.4 AI 工具使用说明与标注范围

本题的问题拆解、Python 与测试草拟、故障定位、图表代码和全部论文文字均使用 OpenAI Codex（GPT-5）辅助 [5]，故本声明适用于上述所有正文与附录位置。核心模型选择、计算结果、证据边界和最终提交责任仍由参赛队员承担；统计数字均由本地程序从只读附件计算并通过独立复跑核验。关键交互、采纳及修改情况另见支撑材料中的《AI 工具使用详情》。

参考文献

[1] 全国大学生数学建模竞赛组织委员会. 2025 年全国大学生数学建模竞赛 C 题[Z]. 2025.

[2] LIANG K Y, ZEGER S L. Longitudinal data analysis using generalized linear models [J/OL]. Biometrika, 1986, 73(1): 13-22. DOI:10.1093/biomet/73.1.13.

[3] ZOU H, HASTIE T. Regularization and variable selection via the elastic net[J/OL]. Journal of the Royal Statistical Society: Series B, 2005, 67(2): 301-320. DOI:10.1111/j.1467-9868.2005.00503.x.

[4] FRIEDMAN J H. Greedy function approximation: a gradient boosting machine [J/OL]. The Annals of Statistics, 2001, 29(5): 1189-1232. DOI:10.1214/aos/1013203451.

[5] OpenAI. Codex, GPT-5, OpenAI, 2026-07-19–2026-07-21[Z]. 2026.

附录A　支撑材料文件列表

支撑材料包括：源程序与测试、冻结运行的输入哈希、环境、阶段报告、结果和产物清单，以及数据说明、模型验证和复现说明。正式分析运行编号与输入哈希见冻结清单。原始 Excel 不打包为派生支撑材料，仍以只读输入哈希引用。

附录B　源程序代码说明

在项目根目录先执行 `scripts/bootstrap_environment.ps1` 建立独立环境，再执行：

```
.\scripts\run_python_file.ps1 `
  -ScriptPath 04_code\run_all.py `
  -JobId c-formal-run-20260721-001 `
  --run-id RUN-C-FORMAL-20260721-001
```

正式入口依次完成数据审计、Q1–Q4、敏感性、图表、运行清单和完整性冻结。论文资产由 `04_code/generate_paper_assets.py` 从冻结运行生成。

附录C　AI 工具使用详情说明

本项目使用 OpenAI Codex 辅助完成题目拆解、代码与测试草拟、错误定位、图表和论文文字整理。所有统计数字均来自本地可复现程序和冻结产物，未由语言模型直接编造；原始附件全程只读。AI 发现并记录了路径、编码、依赖、未收敛拟合、弱基线、Excel 句柄等问题，并以失败测试和回归测试验证修复。AI 未代替参赛队员签署任何人工审批；参赛队员仍需在正式提交前核对竞赛当年格式规范、匿名信息和最终结论。

附录D　完整源程序代码

以下为本题专属、实际执行的 Python 源程序。为便于纸面附录收录，代码采用极小等宽字体；可直接运行的 UTF-8 原文件及通用工作流依赖同时收入支撑材料。

04_code/run_all.py

```
from __future__ import annotations

import argparse
import sys
from pathlib import Path


PROJECT_ROOT = Path(__file__).resolve().parents[1]
REPOSITORY_ROOT = PROJECT_ROOT.parents[1]
for candidate in (PROJECT_ROOT / "04_code", REPOSITORY_ROOT):
    if str(candidate) not in sys.path:
        sys.path.insert(0, str(candidate))

from cumcm2025c.pipeline import run_formal_pipeline


def main() -> int:
    parser = argparse.ArgumentParser(description="Run the complete CUMCM2025-C analysis pipeline.")
    parser.add_argument("--run-id", required=True)
    parser.add_argument("--q2-replicates", type=int, default=200)
    parser.add_argument("--q4-bootstrap-replicates", type=int, default=500)
    parser.add_argument("--run-kind", choices=("analysis", "reproduction"), default="analysis")
    args = parser.parse_args()
    run_dir = run_formal_pipeline(
        PROJECT_ROOT,
        run_id=args.run_id,
        q2_replicates=args.q2_replicates,
        q4_bootstrap_replicates=args.q4_bootstrap_replicates,
        run_kind=args.run_kind,
    )
    print(run_dir)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

04_code/generate_paper_assets.py

```
from __future__ import annotations

import argparse
import sys
from pathlib import Path


PROJECT_ROOT = Path(__file__).resolve().parents[1]
REPOSITORY_ROOT = PROJECT_ROOT.parents[1]
for candidate in (PROJECT_ROOT / "04_code", REPOSITORY_ROOT):
    if str(candidate) not in sys.path:
        sys.path.insert(0, str(candidate))

from cumcm2025c.paper_assets import generate_paper_assets
from cumcm2025c.code_appendix import SOURCE_APPENDIX_FILES, generate_source_appendix


def main() -> int:
    parser = argparse.ArgumentParser(description="Generate LaTeX tables and figures from a frozen C run.")
    parser.add_argument("--run-id", required=True)
    args = parser.parse_args()
    manifest = generate_paper_assets(PROJECT_ROOT, args.run_id)
    generate_source_appendix(PROJECT_ROOT)
    print(f"Generated {len(manifest['artifacts'])} paper assets from {args.run_id}")
    print(f"Generated source appendix from {len(SOURCE_APPENDIX_FILES)} files")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

04_code/compare_runs.py

```
from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path


PROJECT_ROOT = Path(__file__).resolve().parents[1]
REPOSITORY_ROOT = PROJECT_ROOT.parents[1]
for candidate in (PROJECT_ROOT / "04_code", REPOSITORY_ROOT):
    if str(candidate) not in sys.path:
```

```
        sys.path.insert(0, str(candidate))

from cumcm2025c.reproduction_compare import compare_frozen_runs


def main() -> int:
    parser = argparse.ArgumentParser(description="Compare a frozen C analysis run with a frozen reproduction run.")
    parser.add_argument("--reference-run-id", required=True)
    parser.add_argument("--candidate-run-id", required=True)
    parser.add_argument(
        "--output",
        default="10_submission_check/reproduction_report.json",
    )
    args = parser.parse_args()
    report = compare_frozen_runs(PROJECT_ROOT, args.reference_run_id, args.candidate_run_id)
    destination = PROJECT_ROOT / args.output
    destination.parent.mkdir(parents=True, exist_ok=True)
    destination.write_text(
        json.dumps(report, ensure_ascii=False, sort_keys=True, indent=2) + "\n",
        encoding="utf-8",
        newline="\n",
    )
    print(json.dumps({key: value for key, value in report.items() if key != "items"}, ensure_ascii=False, indent=2))
    return 0 if report["status"] == "PASSED" else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

04_code/cumcm2025c/audit.py

```
from __future__ import annotations

import hashlib
import json
from collections import Counter
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd

from .preprocessing import LoadedWorkbook
from .model_data import aggregate_male_technical_events


def _finite_range(series: pd.Series) -> list[float] | None:
    values = pd.to_numeric(series, errors="coerce").dropna()
    if values.empty:
        return None
    return [float(values.min()), float(values.max())]


def _value_counts(series: pd.Series) -> dict[str, int]:
    counts = series.fillna("<MISSING>").astype(str).value_counts(dropna=False)
    return {str(key): int(value) for key, value in counts.items()}


def _frame_audit(frame: pd.DataFrame) -> dict[str, Any]:
    event_sizes = frame.groupby("technical_event_id", sort=False).size()
    event_labels = frame.groupby("technical_event_id", sort=False)["aneuploidy_positive"]
    event_types = frame.groupby("technical_event_id", sort=False)["aneuploidy_types"]
    mother_labels = frame.groupby("mother_code", sort=False)["aneuploidy_positive"].any()

    chromosome_counts: Counter[int] = Counter()
    for labels in frame["aneuploidy_types"]:
        chromosome_counts.update(labels)

    static_fields = ("age", "height_cm", "ivf_mode", "pregnancies", "births")
    varying_static = {
        field: int(
            frame.groupby("mother_code", sort=False)[field]
            .nunique(dropna=False)
            .gt(1)
            .sum()
        )
        for field in static_fields
    }

    y_values = pd.to_numeric(frame["y_concentration"], errors="coerce")
    audit: dict[str, Any] = {
        "rows": int(len(frame)),
        "mothers": int(frame["mother_code"].nunique()),
        "technical_events": int(frame["technical_event_id"].nunique()),
        "technical_repeat_events": int((event_sizes > 1).sum()),
        "rows_in_technical_repeat_events": int(event_sizes[event_sizes > 1].sum()),
        "max_technical_replicates": int(event_sizes.max()),
        "label_inconsistent_technical_events": int(event_labels.nunique().gt(1).sum()),
        "type_inconsistent_technical_events": int(event_types.nunique().gt(1).sum()),
        "positive_aneuploidy_rows": int(frame["aneuploidy_positive"].sum()),
        "positive_aneuploidy_mothers": int(mother_labels.sum()),
        "aneuploidy_chromosome_row_counts": {
            f"T{chromosome}": int(chromosome_counts[chromosome])
            for chromosome in (13, 18, 21)
        },
        "gestational_week_range": _finite_range(frame["gestational_week"]),
        "bmi_range": _finite_range(frame["bmi"]),
        "age_range": _finite_range(frame["age"]),
        "height_cm_range": _finite_range(frame["height_cm"]),
        "weight_kg_range": _finite_range(frame["weight_kg"]),
        "lmp_missing_rows": int(frame["last_menstrual_period"].isna().sum()),
        "bmi_recomputed_rows": int(frame["bmi_was_recomputed"].sum()),
        "gestational_date_gap_over_7_days": int(
            frame["gestational_week_gap_days"].abs().gt(7).sum()
        ),
        "gc_outside_0_40_0_60_rows": int(
```

```
            (~frame["gc_content"].between(0.40, 0.60, inclusive="both")).sum()
        ),
        "mapping_ratio_range": _finite_range(frame["mapping_ratio"]),
        "duplicate_ratio_range": _finite_range(frame["duplicate_ratio"]),
        "filtered_ratio_range": _finite_range(frame["filtered_ratio"]),
        "fetal_health_counts": _value_counts(frame["fetal_healthy"]),
        "ivf_mode_counts": _value_counts(frame["ivf_mode"]),
        "rows_per_mother": {
            "min": int(frame.groupby("mother_code").size().min()),
            "median": float(frame.groupby("mother_code").size().median()),
            "max": int(frame.groupby("mother_code").size().max()),
        },
        "mothers_with_varying_nominally_static_fields": varying_static,
        "y_concentration_range": _finite_range(y_values),
        "y_concentration_at_least_0_04_rows": (
            int(y_values.ge(0.04).sum()) if y_values.notna().any() else None
        ),
    }
    return audit


def build_data_audit(loaded: LoadedWorkbook) -> dict[str, Any]:
    return {
        "schema_version": "1.0",
        "source_path": loaded.source_path,
        "source_sha256": loaded.source_sha256,
        "sheet_names": list(loaded.sheet_names),
        "male": _frame_audit(loaded.male),
        "female": _frame_audit(loaded.female),
    }


def _sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def _export_frame(frame: pd.DataFrame, path: Path) -> None:
    exported = frame.copy()
    if "aneuploidy_types" in exported:
        exported["aneuploidy_types"] = exported["aneuploidy_types"].map(
            lambda values: "|".join(f"T{value}" for value in values)
        )
    exported.to_csv(
        path,
        index=False,
        encoding="utf-8-sig",
        lineterminator="\n",
        date_format="%Y-%m-%d",
        float_format="%.10g",
    )


def write_processed_bundle(loaded: LoadedWorkbook, output_dir: str | Path) -> dict[str, Any]:
    destination = Path(output_dir)
    destination.mkdir(parents=True, exist_ok=True)

    male_path = destination / "male_clean.csv"
    female_path = destination / "female_clean.csv"
    male_events_path = destination / "male_events.csv"
    audit_path = destination / "data_audit.json"
    manifest_path = destination / "manifest.json"

    _export_frame(loaded.male, male_path)
    _export_frame(loaded.female, female_path)
    male_events = aggregate_male_technical_events(loaded.male)
    _export_frame(male_events, male_events_path)
    audit_path.write_text(
        json.dumps(build_data_audit(loaded), ensure_ascii=False, sort_keys=True, indent=2)
        + "\n",
        encoding="utf-8",
        newline="\n",
    )

    artifacts: dict[str, dict[str, Any]] = {}
    for path, rows in (
        (male_path, len(loaded.male)),
        (female_path, len(loaded.female)),
        (male_events_path, len(male_events)),
        (audit_path, None),
    ):
        metadata: dict[str, Any] = {"sha256": _sha256(path), "bytes": path.stat().st_size}
        if rows is not None:
            metadata["rows"] = int(rows)
        artifacts[path.name] = metadata

    manifest: dict[str, Any] = {
        "schema_version": "1.0",
        "source_sha256": loaded.source_sha256,
        "artifacts": artifacts,
    }
    manifest_path.write_text(
        json.dumps(manifest, ensure_ascii=False, sort_keys=True, indent=2) + "\n",
        encoding="utf-8",
        newline="\n",
    )
    return manifest
```

04_code/cumcm2025c/model_data.py

```
from __future__ import annotations

import math
```

```
import re
from collections.abc import Iterator

import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedGroupKFold


_LOWER_BOUNDED_RE = re.compile(r"^(?:>=|>=)\s*(\d+(?:\.\d+)?)$")


def parse_lower_bounded_count(raw: object) -> tuple[float, bool]:
    if raw is None or (not isinstance(raw, str) and pd.isna(raw)):
        return (math.nan, False)
    if isinstance(raw, str):
        text = raw.strip()
        match = _LOWER_BOUNDED_RE.fullmatch(text)
        if match:
            return (float(match.group(1)), True)
        try:
            return (float(text), False)
        except ValueError as exc:
            raise ValueError(f"Invalid count value: {raw!r}") from exc
    try:
        return (float(raw), False)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"Invalid count value: {raw!r}") from exc


def aggregate_male_technical_events(male: pd.DataFrame) -> pd.DataFrame:
    required = {
        "technical_event_id",
        "mother_code",
        "exam_date",
        "draw_number",
        "gestational_week",
        "bmi",
        "y_concentration",
    }
    missing = required.difference(male.columns)
    if missing:
        raise ValueError(f"Male frame is missing required columns: {sorted(missing)}")

    numeric_median = (
        "age",
        "height_cm",
        "weight_kg",
        "draw_number",
        "gestational_week",
        "bmi",
        "raw_reads",
        "mapping_ratio",
        "duplicate_ratio",
        "unique_mapped_reads",
        "gc_content",
        "z13",
        "z18",
        "z21",
        "x_zscore",
        "y_zscore",
        "y_concentration",
        "x_concentration",
        "gc13",
        "gc18",
        "gc21",
        "filtered_ratio",
        "births",
    )
    aggregation: dict[str, str] = {
        column: "median" for column in numeric_median if column in male.columns
    }
    aggregation.update(
        {
            "mother_code": "first",
            "exam_date": "min",
            "gestational_week_raw": "first",
            "ivf_mode": "first",
            "pregnancies": "first",
            "fetal_healthy": "first",
        }
    )

    grouped = male.groupby("technical_event_id", sort=False, observed=True)
    events = grouped.agg(aggregation).reset_index()
    event_sizes = grouped.size()
    y_sd = grouped["y_concentration"].std(ddof=1).fillna(0.0)
    events["technical_replicate_count"] = events["technical_event_id"].map(event_sizes).astype(int)
    events["y_technical_sd"] = events["technical_event_id"].map(y_sd).astype(float)
    events["y_attained"] = events["y_concentration"].ge(0.04)

    pregnancy_parsed = events["pregnancies"].map(parse_lower_bounded_count)
    events["pregnancies_numeric"] = pregnancy_parsed.map(lambda value: value[0])
    events["pregnancies_lower_bounded"] = pregnancy_parsed.map(lambda value: value[1])
    events["ivf_assisted"] = events["ivf_mode"].astype(str).ne("自然受孕")

    events = events.sort_values(
        ["mother_code", "exam_date", "draw_number", "technical_event_id"],
        kind="mergesort",
    ).reset_index(drop=True)
    mother_group = events.groupby("mother_code", sort=False)
    events["mother_event_order"] = mother_group.cumcount() + 1
    events["mother_event_count"] = mother_group["technical_event_id"].transform("size")
    return events


def stratified_mother_splits(
    target: np.ndarray | pd.Series,
    groups: np.ndarray | pd.Series,
    *,
    n_splits: int,
```

```
    random_state: int,
) -> Iterator[tuple[np.ndarray, np.ndarray]]:
    y = np.asarray(target, dtype=int)
    group_values = np.asarray(groups)
    if y.ndim != 1 or group_values.ndim != 1 or len(y) != len(group_values):
        raise ValueError("target and groups must be one-dimensional arrays of equal length")
    if len(np.unique(group_values)) < n_splits:
        raise ValueError("n_splits exceeds the number of unique mothers")

    splitter = StratifiedGroupKFold(
        n_splits=n_splits,
        shuffle=True,
        random_state=random_state,
    )
    features = np.zeros((len(y), 1), dtype=float)
    for train_idx, test_idx in splitter.split(features, y, group_values):
        train_groups = set(group_values[train_idx])
        test_groups = set(group_values[test_idx])
        if not train_groups.isdisjoint(test_groups):
            raise RuntimeError("Mother leakage detected in grouped split")
        yield train_idx, test_idx
```

04_code/cumcm2025c/preprocessing.py

```
from __future__ import annotations

import hashlib
import math
import numbers
import re
from dataclasses import dataclass
from datetime import date, datetime
from pathlib import Path
from typing import Final

import pandas as pd


SOURCE_SHA256: Final[str] = "14827156218bd4f7e4f16db4aa6d9f757c6648379e038ae6c6b58383648614af"

CANONICAL_COLUMNS: Final[tuple[str, ...]] = (
    "serial_no",
    "mother_code",
    "age",
    "height_cm",
    "weight_kg",
    "last_menstrual_period",
    "ivf_mode",
    "exam_date",
    "draw_number",
    "gestational_week_raw",
    "bmi",
    "raw_reads",
    "mapping_ratio",
    "duplicate_ratio",
    "unique_mapped_reads",
    "gc_content",
    "z13",
    "z18",
    "z21",
    "x_zscore",
    "y_zscore",
    "y_concentration",
    "x_concentration",
    "gc13",
    "gc18",
    "gc21",
    "filtered_ratio",
    "aneuploidy_raw",
    "pregnancies",
    "births",
    "fetal_healthy",
)

_GESTATIONAL_WEEK_RE = re.compile(r"^\s*(\d+)[wW](?:\+(\d+))?\s*$")
_ANEUPLOIDY_RE = re.compile(r"T?(13|18|21)", flags=re.IGNORECASE)


@dataclass(frozen=True)
class LoadedWorkbook:
    source_path: str
    source_sha256: str
    sheet_names: tuple[str, str]
    male: pd.DataFrame
    female: pd.DataFrame


def file_sha256(path: str | Path) -> str:
    digest = hashlib.sha256()
    with Path(path).open("rb") as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def parse_gestational_week(raw: object) -> float:
    match = _GESTATIONAL_WEEK_RE.fullmatch(str(raw)) if raw is not None else None
    if match is None:
        raise ValueError(f"Invalid gestational week: {raw!r}")
    weeks = int(match.group(1))
    days = int(match.group(2) or 0)
    if days not in range(7):
        raise ValueError(f"Gestational days must be between 0 and 6: {raw!r}")
    return weeks + days / 7.0
```

```
def normalize_exam_date(raw: object) -> pd.Timestamp:
    if isinstance(raw, (pd.Timestamp, datetime, date)):
        return pd.Timestamp(raw).normalize()

    if isinstance(raw, numbers.Real) and not isinstance(raw, bool):
        if not math.isfinite(float(raw)) or not float(raw).is_integer():
            raise ValueError(f"Invalid numeric exam date: {raw!r}")
        raw = str(int(raw))
    elif isinstance(raw, str):
        raw = raw.strip()
    else:
        raise ValueError(f"Unsupported exam date type: {type(raw).__name__}")

    parsed = pd.to_datetime(raw, format="%Y%m%d", errors="coerce")
    if pd.isna(parsed):
        raise ValueError(f"Invalid exam date: {raw!r}")
    return pd.Timestamp(parsed).normalize()


def parse_aneuploidy(raw: object) -> tuple[int, ...]:
    if raw is None or (not isinstance(raw, str) and pd.isna(raw)):
        return ()
    text = str(raw).strip().upper()
    if not text:
        return ()
    chromosomes = tuple(sorted({int(value) for value in _ANEUPLOIDY_RE.findall(text)}))
    if not chromosomes:
        raise ValueError(f"Unknown aneuploidy label: {raw!r}")
    return chromosomes


def _normalize_lmp(raw: object) -> pd.Timestamp | pd.NaT:
    if raw is None or pd.isna(raw):
        return pd.NaT
    if isinstance(raw, (pd.Timestamp, datetime, date)):
        return pd.Timestamp(raw).normalize()
    parsed = pd.to_datetime(str(raw).strip(), errors="coerce")
    return pd.NaT if pd.isna(parsed) else pd.Timestamp(parsed).normalize()


def _canonicalize_frame(frame: pd.DataFrame, sex: str) -> pd.DataFrame:
    if frame.shape[1] != len(CANONICAL_COLUMNS):
        raise ValueError(
            f"Expected {len(CANONICAL_COLUMNS)} columns for {sex}, found {frame.shape[1]}"
        )

    result = frame.copy(deep=True)
    result.columns = CANONICAL_COLUMNS
    result["mother_code"] = result["mother_code"].astype("string").str.strip()
    result["exam_date"] = result["exam_date"].map(normalize_exam_date)
    result["last_menstrual_period"] = result["last_menstrual_period"].map(_normalize_lmp)
    result["gestational_week"] = result["gestational_week_raw"].map(parse_gestational_week)

    result["bmi_reported"] = result["bmi"]
    calculated_bmi = result["weight_kg"] / (result["height_cm"] / 100.0) ** 2
    result["bmi_was_recomputed"] = result["bmi"].isna()
    result.loc[result["bmi_was_recomputed"], "bmi"] = calculated_bmi[result["bmi_was_recomputed"]]
    result["bmi_calculated"] = calculated_bmi
    result["bmi_absolute_difference"] = (result["bmi"] - calculated_bmi).abs()

    result["gestational_week_from_dates"] = (
        result["exam_date"] - result["last_menstrual_period"]
    ).dt.days / 7.0
    result["gestational_week_gap_days"] = (
        result["gestational_week"] - result["gestational_week_from_dates"]
    ) * 7.0

    result["aneuploidy_types"] = result["aneuploidy_raw"].map(parse_aneuploidy)
    result["aneuploidy_positive"] = result["aneuploidy_types"].map(bool)

    event_date = result["exam_date"].dt.strftime("%Y%m%d")
    result["technical_event_id"] = (
        result["mother_code"].astype(str)
        + "|"
        + event_date
        + "|"
        + result["draw_number"].astype("Int64").astype(str)
    )
    grouped = result.groupby("technical_event_id", sort=False, dropna=False)
    result["technical_replicate_index"] = grouped.cumcount() + 1
    result["technical_replicate_count"] = grouped["technical_event_id"].transform("size")
    result["sex"] = sex
    return result


def load_official_workbook(path: str | Path) -> LoadedWorkbook:
    source = Path(path).resolve()
    if not source.is_file():
        raise FileNotFoundError(source)
    source_hash = file_sha256(source)
    if source_hash != SOURCE_SHA256:
        raise ValueError(
            "Workbook SHA256 does not match the registered official attachment: "
            f"expected {SOURCE_SHA256}, got {source_hash}"
        )

    with pd.ExcelFile(source, engine="openpyxl") as excel:
        if len(excel.sheet_names) != 2:
            raise ValueError(f"Expected exactly two worksheets, found {excel.sheet_names!r}")
        sheet_names = (excel.sheet_names[0], excel.sheet_names[1])
        male_raw = pd.read_excel(excel, sheet_name=sheet_names[0])
        female_raw = pd.read_excel(excel, sheet_name=sheet_names[1])

    male = _canonicalize_frame(male_raw, sex="male")
    female = _canonicalize_frame(female_raw, sex="female")
    return LoadedWorkbook(
        source_path=str(source),
        source_sha256=source_hash,
        sheet_names=sheet_names,
        male=male,
```

```
        female=female,
    )
```

04_code/cumcm2025c/q1_longitudinal.py

```
from __future__ import annotations

import hashlib
import json
from dataclasses import dataclass
from pathlib import Path
from typing import Any

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from patsy import build_design_matrices
from scipy.special import expit
from scipy.stats import chi2, probplot


STANDARDIZED_COLUMNS = (
    "bmi",
    "age",
    "height_cm",
    "gc_content",
    "mapping_ratio",
    "duplicate_ratio",
    "filtered_ratio",
)

SPLINE_TERM = "bs(gestational_week, df=4, degree=3, include_intercept=False)"
QUALITY_TERMS = "gc_content_z + mapping_ratio_z + duplicate_ratio_z + filtered_ratio_z"
MATERNAL_TERMS = "age_z + height_cm_z + ivf_assisted"
COMMON_TERMS = f"{MATERNAL_TERMS} + {QUALITY_TERMS}"

FORMULAS = {
    "linear_baseline": f"y_logit ~ gestational_week + bmi_z + {COMMON_TERMS}",
    "interaction_candidate": f"y_logit ~ {SPLINE_TERM} * bmi_z + {COMMON_TERMS}",
    "selected": f"y_logit ~ {SPLINE_TERM} + bmi_z + {COMMON_TERMS}",
    "without_gestational_week": f"y_logit ~ bmi_z + {COMMON_TERMS}",
    "without_bmi": f"y_logit ~ {SPLINE_TERM} + {COMMON_TERMS}",
    "without_quality": f"y_logit ~ {SPLINE_TERM} + bmi_z + {MATERNAL_TERMS}",
}


@dataclass
class Q1ModelResults:
    fits: dict[str, Any]
    selected_model: str
    robustness_fit: Any
    robustness_tests: dict[str, dict[str, float | int]]
    block_tests: dict[str, dict[str, float | int]]
    metrics: dict[str, float]
    coefficients: pd.DataFrame


def prepare_q1_data(events: pd.DataFrame) -> pd.DataFrame:
    data = events.copy(deep=True)
    if not data["y_concentration"].between(0, 1, inclusive="neither").all():
        raise ValueError("Y concentration must lie strictly between zero and one")
    data.loc[:, "y_logit"] = np.log(
        data["y_concentration"] / (1.0 - data["y_concentration"])
    )

    scaling: dict[str, dict[str, float]] = {}
    for column in STANDARDIZED_COLUMNS:
        values = pd.to_numeric(data[column], errors="raise").astype(float)
        mean = float(values.mean())
        standard_deviation = float(values.std(ddof=0))
        if not np.isfinite(standard_deviation) or standard_deviation <= 0:
            raise ValueError(f"Cannot standardize constant or invalid column: {column}")
        data.loc[:, f"{column}_z"] = (values - mean) / standard_deviation
        scaling[column] = {"mean": mean, "std": standard_deviation}

    data = data.assign(ivf_assisted=data["ivf_assisted"].astype(int))
    required = [
        "y_logit",
        "mother_code",
        "gestational_week",
        *(f"{column}_z" for column in STANDARDIZED_COLUMNS),
        "ivf_assisted",
    ]
    if data.loc[:, required].isna().any().any():
        missing = data.loc[:, required].isna().sum().loc[lambda values: values.gt(0)]
        raise ValueError(f"Q1 data contain missing model fields: {missing.to_dict()}")
    data.attrs["standardization"] = scaling
    return data


def _fit_mixed_model(formula: str, data: pd.DataFrame):
    model = smf.mixedlm(
        formula,
        data,
        groups=data["mother_code"],
        re_formula="1",
        missing="raise",
    )
    fit = model.fit(reml=False, method="lbfgs", maxiter=1000, disp=False)
    if not fit.converged:
        fit = model.fit(reml=False, method="powell", maxiter=2000, disp=False)
```

```
    return fit


def _likelihood_ratio(full_fit: Any, reduced_fit: Any) -> dict[str, float | int]:
    degrees_of_freedom = int(len(full_fit.fe_params) - len(reduced_fit.fe_params))
    if degrees_of_freedom < 1:
        raise ValueError("Reduced model must have fewer fixed-effect parameters")
    statistic = max(0.0, float(2.0 * (full_fit.llf - reduced_fit.llf)))
    return {
        "lr_statistic": statistic,
        "degrees_of_freedom": degrees_of_freedom,
        "p_value": float(chi2.sf(statistic, degrees_of_freedom)),
    }


def _coefficient_table(fit: Any) -> pd.DataFrame:
    names = list(fit.fe_params.index)
    estimates = fit.fe_params.loc[names].astype(float)
    standard_errors = fit.bse_fe.loc[names].astype(float)
    table = pd.DataFrame(
        {
            "term": names,
            "estimate": estimates.to_numpy(),
            "standard_error": standard_errors.to_numpy(),
            "lower_95": (estimates - 1.96 * standard_errors).to_numpy(),
            "upper_95": (estimates + 1.96 * standard_errors).to_numpy(),
            "p_value": fit.pvalues.loc[names].astype(float).to_numpy(),
        }
    )
    return table


def _fit_robust_gee(data: pd.DataFrame):
    model = smf.gee(
        FORMULAS["selected"],
        groups="mother_code",
        data=data,
        family=sm.families.Gaussian(),
        cov_struct=sm.cov_struct.Exchangeable(),
        missing="raise",
    )
    return model.fit(maxiter=1000)


def _gee_wald_block(fit: Any, names: list[str]) -> dict[str, float | int]:
    beta = fit.params.loc[names].to_numpy(dtype=float)
    covariance = fit.cov_params().loc[names, names].to_numpy(dtype=float)
    statistic = float(beta @ np.linalg.pinv(covariance) @ beta)
    degrees_of_freedom = len(names)
    return {
        "wald_statistic": statistic,
        "degrees_of_freedom": degrees_of_freedom,
        "p_value": float(chi2.sf(statistic, degrees_of_freedom)),
    }


def _gee_coefficient_table(fit: Any) -> pd.DataFrame:
    estimates = fit.params.astype(float)
    standard_errors = fit.bse.astype(float)
    return pd.DataFrame(
        {
            "term": list(estimates.index),
            "estimate": estimates.to_numpy(),
            "robust_standard_error": standard_errors.to_numpy(),
            "lower_95": (estimates - 1.96 * standard_errors).to_numpy(),
            "upper_95": (estimates + 1.96 * standard_errors).to_numpy(),
            "p_value": fit.pvalues.astype(float).to_numpy(),
        }
    )


def fit_q1_models(data: pd.DataFrame) -> Q1ModelResults:
    fits = {name: _fit_mixed_model(formula, data) for name, formula in FORMULAS.items()}
    selected = fits["selected"]
    interaction_candidate = fits["interaction_candidate"]
    block_tests = {
        "gestational_week": _likelihood_ratio(selected, fits["without_gestational_week"]),
        "bmi": _likelihood_ratio(selected, fits["without_bmi"]),
        "week_bmi_interaction": _likelihood_ratio(interaction_candidate, selected),
        "quality": _likelihood_ratio(selected, fits["without_quality"]),
    }
    robustness_fit = _fit_robust_gee(data)
    week_names = [name for name in robustness_fit.params.index if name.startswith("bs(")]
    robustness_tests = {
        "gestational_week": _gee_wald_block(robustness_fit, week_names),
        "bmi": _gee_wald_block(robustness_fit, ["bmi_z"]),
    }

    fixed_prediction = np.asarray(selected.model.exog) @ np.asarray(selected.fe_params)
    fixed_variance = float(np.var(fixed_prediction, ddof=1))
    random_variance = float(selected.cov_re.iloc[0, 0])
    residual_variance = float(selected.scale)
    total_variance = fixed_variance + random_variance + residual_variance
    fitted = np.asarray(selected.fittedvalues, dtype=float)
    observed = np.asarray(selected.model.endog, dtype=float)
    metrics = {
        "n_events": float(len(data)),
        "n_mothers": float(data["mother_code"].nunique()),
        "linear_baseline_aic": float(fits["linear_baseline"].aic),
        "interaction_candidate_aic": float(interaction_candidate.aic),
        "selected_aic": float(selected.aic),
        "selected_bic": float(selected.bic),
        "log_likelihood": float(selected.llf),
        "random_intercept_variance": random_variance,
        "residual_variance": residual_variance,
        "intraclass_correlation": random_variance / (random_variance + residual_variance),
        "marginal_r2": fixed_variance / total_variance,
        "conditional_r2": (fixed_variance + random_variance) / total_variance,
        "conditional_rmse_logit": float(np.sqrt(np.mean((observed - fitted) ** 2))),
        "gee_exchangeable_correlation": float(robustness_fit.cov_struct.dep_params),
```

```
        "gee_bmi_estimate": float(robustness_fit.params["bmi_z"]),
        "gee_bmi_p_value": float(robustness_fit.pvalues["bmi_z"]),
    }
    return Q1ModelResults(
        fits=fits,
        selected_model="selected",
        robustness_fit=robustness_fit,
        robustness_tests=robustness_tests,
        block_tests=block_tests,
        metrics=metrics,
        coefficients=_coefficient_table(selected),
    )


def predict_q1_curves(
    data: pd.DataFrame,
    fit: Any,
    *,
    week_step: float = 0.25,
) -> pd.DataFrame:
    if week_step <= 0:
        raise ValueError("week_step must be positive")
    scaling = data.attrs.get("standardization")
    if not scaling:
        raise ValueError("Prepared Q1 data are missing standardization metadata")

    bmi_values = np.quantile(data["bmi"], [0.25, 0.50, 0.75])
    labels = ("P25", "P50", "P75")
    weeks = np.arange(11.0, 25.0 + week_step / 2.0, week_step)
    records: list[dict[str, float | str | int]] = []
    for label, bmi in zip(labels, bmi_values, strict=True):
        bmi_z = (float(bmi) - scaling["bmi"]["mean"]) / scaling["bmi"]["std"]
        for week in weeks:
            records.append(
                {
                    "bmi_level": label,
                    "bmi": float(bmi),
                    "bmi_z": float(bmi_z),
                    "gestational_week": float(week),
                    "age_z": 0.0,
                    "height_cm_z": 0.0,
                    "gc_content_z": 0.0,
                    "mapping_ratio_z": 0.0,
                    "duplicate_ratio_z": 0.0,
                    "filtered_ratio_z": 0.0,
                    "ivf_assisted": 0,
                }
            )
    prediction_frame = pd.DataFrame.from_records(records)

    design_info = fit.model.data.design_info
    design = np.asarray(build_design_matrices([design_info], prediction_frame)[0], dtype=float)
    fixed_names = list(fit.fe_params.index)
    covariance = fit.cov_params().loc[fixed_names, fixed_names].to_numpy(dtype=float)
    linear_prediction = design @ fit.fe_params.to_numpy(dtype=float)
    standard_error = np.sqrt(np.einsum("ij,jk,ik->i", design, covariance, design))

    prediction_frame.loc[:, "predicted_y"] = expit(linear_prediction)
    prediction_frame.loc[:, "lower_95"] = expit(linear_prediction - 1.96 * standard_error)
    prediction_frame.loc[:, "upper_95"] = expit(linear_prediction + 1.96 * standard_error)
    return prediction_frame[
        ["bmi_level", "bmi", "gestational_week", "predicted_y", "lower_95", "upper_95"]
    ]


def _save_figure(fig: plt.Figure, output_dir: Path, stem: str) -> None:
    fig.savefig(
        output_dir / f"{stem}.png",
        dpi=300,
        bbox_inches="tight",
        facecolor="white",
        metadata={"Software": "CUMCM2025-C reproducible pipeline"},
    )
    fig.savefig(
        output_dir / f"{stem}.pdf",
        bbox_inches="tight",
        facecolor="white",
        metadata={"Creator": "CUMCM2025-C reproducible pipeline"},
    )
    plt.close(fig)


def _plot_relationship(curves: pd.DataFrame, output_dir: Path) -> None:
    fig, axis = plt.subplots(figsize=(7.2, 4.8))
    colors = {"P25": "#1677B3", "P50": "#E07A1F", "P75": "#2A9D63"}
    for label, group in curves.groupby("bmi_level", sort=False):
        x = group["gestational_week"].to_numpy(dtype=float)
        y = group["predicted_y"].to_numpy(dtype=float) * 100.0
        lower = group["lower_95"].to_numpy(dtype=float) * 100.0
        upper = group["upper_95"].to_numpy(dtype=float) * 100.0
        bmi = float(group["bmi"].iloc[0])
        axis.plot(x, y, color=colors[label], linewidth=2.2, label=f"BMI {bmi:.1f} ({label})")
        axis.fill_between(x, lower, upper, color=colors[label], alpha=0.15, linewidth=0)
    axis.axhline(4.0, color="#B22222", linestyle="--", linewidth=1.4, label="4% threshold")
    axis.set(xlabel="Gestational week", ylabel="Predicted Y concentration (%)")
    axis.set_xlim(11, 25)
    axis.grid(alpha=0.22)
    axis.legend(frameon=False, ncol=2)
    fig.tight_layout()
    _save_figure(fig, output_dir, "q1_relationship")


def _plot_observed(data: pd.DataFrame, output_dir: Path) -> None:
    fig, axis = plt.subplots(figsize=(7.2, 4.8))
    scatter = axis.scatter(
        data["gestational_week"],
        data["y_concentration"] * 100.0,
        c=data["bmi"],
        cmap="viridis",
        s=18,
```

```
        alpha=0.58,
        linewidths=0,
    )
    axis.axhline(4.0, color="#B22222", linestyle="--", linewidth=1.4)
    axis.set(xlabel="Gestational week", ylabel="Observed Y concentration (%)")
    axis.grid(alpha=0.18)
    colorbar = fig.colorbar(scatter, ax=axis, pad=0.02)
    colorbar.set_label("BMI")
    fig.tight_layout()
    _save_figure(fig, output_dir, "q1_observed")


def _plot_diagnostics(data: pd.DataFrame, fit: Any, output_dir: Path) -> None:
    residuals = np.asarray(fit.resid, dtype=float)
    fitted_logit = np.asarray(fit.fittedvalues, dtype=float)
    fitted_y = expit(fitted_logit)
    theoretical, ordered = probplot(residuals, dist="norm", fit=False)
    slope, intercept = np.polyfit(np.asarray(theoretical), np.asarray(ordered), deg=1)
    random_intercepts = np.asarray(
        [float(np.asarray(value, dtype=float)[0]) for value in fit.random_effects.values()]
    )

    fig, axes = plt.subplots(2, 2, figsize=(9.2, 7.2))
    axes[0, 0].scatter(fitted_logit, residuals, s=15, alpha=0.48, linewidths=0)
    axes[0, 0].axhline(0, color="#B22222", linestyle="--", linewidth=1.1)
    axes[0, 0].set(xlabel="Conditional fitted logit", ylabel="Residual")

    axes[0, 1].scatter(theoretical, ordered, s=15, alpha=0.55, linewidths=0)
    theory = np.asarray(theoretical, dtype=float)
    axes[0, 1].plot(theory, intercept + slope * theory, color="#B22222", linewidth=1.2)
    axes[0, 1].set(xlabel="Normal theoretical quantile", ylabel="Residual quantile")

    axes[1, 0].hist(random_intercepts, bins=22, color="#3F7CAC", alpha=0.82, edgecolor="white")
    axes[1, 0].set(xlabel="Mother random intercept", ylabel="Count")

    axes[1, 1].scatter(
        data["y_concentration"] * 100.0,
        fitted_y * 100.0,
        s=15,
        alpha=0.48,
        linewidths=0,
    )
    maximum = float(max(data["y_concentration"].max(), fitted_y.max()) * 100.0)
    axes[1, 1].plot([0, maximum], [0, maximum], color="#B22222", linestyle="--", linewidth=1.1)
    axes[1, 1].set(xlabel="Observed Y concentration (%)", ylabel="Conditional fitted (%)")

    for axis in axes.flat:
        axis.grid(alpha=0.16)
    fig.tight_layout()
    _save_figure(fig, output_dir, "q1_diagnostics")


def _file_sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def write_q1_outputs(
    data: pd.DataFrame,
    results: Q1ModelResults,
    output_dir: str | Path,
) -> dict[str, Any]:
    destination = Path(output_dir)
    destination.mkdir(parents=True, exist_ok=True)
    selected_fit = results.fits[results.selected_model]
    curves = predict_q1_curves(data, selected_fit)

    metrics_payload = {
        "schema_version": "1.0",
        "selected_model": results.selected_model,
        "selected_formula": FORMULAS[results.selected_model],
        "interaction_candidate_formula": FORMULAS["interaction_candidate"],
        "metrics": results.metrics,
        "robustness_tests": results.robustness_tests,
    }
    (destination / "q1_metrics.json").write_text(
        json.dumps(metrics_payload, ensure_ascii=False, sort_keys=True, indent=2) + "\n",
        encoding="utf-8",
        newline="\n",
    )

    block_table = pd.DataFrame.from_dict(results.block_tests, orient="index").reset_index(
        names="effect_block"
    )
    block_table.to_csv(destination / "q1_block_tests.csv", index=False, lineterminator="\n")
    results.coefficients.to_csv(
        destination / "q1_coefficients.csv", index=False, lineterminator="\n"
    )
    _gee_coefficient_table(results.robustness_fit).to_csv(
        destination / "q1_gee_coefficients.csv", index=False, lineterminator="\n"
    )
    curves.to_csv(destination / "q1_curves.csv", index=False, lineterminator="\n")

    comparison = pd.DataFrame(
        [
            {
                "model": name,
                "formula": FORMULAS[name],
                "converged": bool(fit.converged),
                "log_likelihood": float(fit.llf),
                "aic": float(fit.aic),
                "bic": float(fit.bic),
                "fixed_effect_parameters": int(len(fit.fe_params)),
            }
            for name, fit in results.fits.items()
        ]
    )
```

```
    comparison.to_csv(destination / "q1_model_comparison.csv", index=False, lineterminator="\n")
    (destination / "q1_model_summary.txt").write_text(
        selected_fit.summary().as_text() + "\n", encoding="utf-8", newline="\n"
    )

    _plot_relationship(curves, destination)
    _plot_observed(data, destination)
    _plot_diagnostics(data, selected_fit, destination)

    artifacts: dict[str, dict[str, int | str]] = {}
    for path in sorted(destination.iterdir()):
        if path.is_file() and path.name != "q1_manifest.json":
            artifacts[path.name] = {"sha256": _file_sha256(path), "bytes": path.stat().st_size}
    manifest = {
        "schema_version": "1.0",
        "selected_model": results.selected_model,
        "artifacts": artifacts,
    }
    (destination / "q1_manifest.json").write_text(
        json.dumps(manifest, ensure_ascii=False, sort_keys=True, indent=2) + "\n",
        encoding="utf-8",
        newline="\n",
    )
    return manifest
```

04_code/cumcm2025c/q2_timing.py

```
from __future__ import annotations

import hashlib
import json
import warnings
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Iterable

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy.special import expit
from scipy.stats import chi2
from statsmodels.tools.sm_exceptions import IterationLimitWarning


@dataclass
class Q2AttainmentModel:
    selected_fit: Any
    interaction_fit: Any
    interaction_p_value: float
    interaction_converged: bool


@dataclass
class Q2Policy:
    n_groups: int
    groups: list[dict[str, float | int | bool]]
    total_regret_weeks: float
    mean_regret_weeks: float
    complexity_penalty_weeks: float
    penalized_objective: float


@dataclass
class Q2Analysis:
    prepared_data: pd.DataFrame
    model: Q2AttainmentModel
    profiles: pd.DataFrame
    individual: pd.DataFrame
    policy: Q2Policy
    group_count_tradeoff: pd.DataFrame
    measurement_error: dict[str, float | int]
    reliability_target: float
    y_threshold: float
    minimum_group_size: int
    complexity_penalty_weeks: float


def prepare_q2_data(events: pd.DataFrame, *, y_threshold: float = 0.04) -> pd.DataFrame:
    if not 0 < y_threshold < 1:
        raise ValueError("y_threshold must lie between zero and one")
    data = events.copy(deep=True)
    data = data.assign(
        attained=data["y_concentration"].ge(y_threshold).astype(int),
        week_centered=data["gestational_week"].astype(float) - 15.0,
    )
    bmi_mean = float(data["bmi"].mean())
    bmi_standard_deviation = float(data["bmi"].std(ddof=0))
    if bmi_standard_deviation <= 0:
        raise ValueError("BMI has no variation")
    data = data.assign(bmi_z=(data["bmi"] - bmi_mean) / bmi_standard_deviation)
    required = ["attained", "week_centered", "bmi_z", "mother_code"]
    if data[required].isna().any().any():
        raise ValueError("Q2 model data contain missing required values")
    data.attrs["bmi_scaling"] = {"mean": bmi_mean, "std": bmi_standard_deviation}
    data.attrs["y_threshold"] = float(y_threshold)
    return data


def _fit_q2_gee(data: pd.DataFrame, formula: str):
    model = smf.gee(
        formula,
```

```
        groups="mother_code",
        data=data,
        family=sm.families.Binomial(),
        cov_struct=sm.cov_struct.Exchangeable(),
        missing="raise",
    )
    with warnings.catch_warnings():
        warnings.simplefilter("ignore", IterationLimitWarning)
        return model.fit(maxiter=1000)


def _fit_q2_selected(data: pd.DataFrame):
    return _fit_q2_gee(data, "attained ~ week_centered + bmi_z")


def fit_q2_attainment_model(data: pd.DataFrame) -> Q2AttainmentModel:
    selected = _fit_q2_selected(data)
    if not selected.converged:
        raise RuntimeError("Selected Q2 GEE did not converge")
    interaction = _fit_q2_gee(data, "attained ~ week_centered * bmi_z")
    interaction_p = (
        float(interaction.pvalues["week_centered:bmi_z"])
        if interaction.converged
        else float("nan")
    )
    return Q2AttainmentModel(
        selected_fit=selected,
        interaction_fit=interaction,
        interaction_p_value=interaction_p,
        interaction_converged=bool(interaction.converged),
    )


def predict_attainment_probability(
    fit: Any,
    prepared_data: pd.DataFrame,
    gestational_weeks: np.ndarray | Iterable[float],
    bmi_values: np.ndarray | Iterable[float],
) -> np.ndarray:
    weeks = np.asarray(gestational_weeks, dtype=float)
    bmi = np.asarray(bmi_values, dtype=float)
    weeks, bmi = np.broadcast_arrays(weeks, bmi)
    scaling = prepared_data.attrs.get("bmi_scaling")
    if not scaling:
        raise ValueError("Prepared Q2 data are missing BMI scaling metadata")
    week_centered = weeks - 15.0
    bmi_z = (bmi - scaling["mean"]) / scaling["std"]
    linear = (
        float(fit.params["Intercept"])
        + float(fit.params["week_centered"]) * week_centered
        + float(fit.params["bmi_z"]) * bmi_z
    )
    return expit(linear)


def baseline_mother_profiles(events: pd.DataFrame) -> pd.DataFrame:
    ordered = events.sort_values(
        ["mother_code", "exam_date", "draw_number", "technical_event_id"],
        kind="mergesort",
    )
    profiles = ordered.groupby("mother_code", sort=False, as_index=False).first()
    return profiles[
        [
            "mother_code",
            "bmi",
            "age",
            "height_cm",
            "weight_kg",
            "ivf_assisted",
            "pregnancies_numeric",
            "pregnancies_lower_bounded",
            "births",
        ]
    ].copy()


def individual_recommended_weeks(
    profiles: pd.DataFrame,
    fit: Any,
    prepared_data: pd.DataFrame,
    *,
    reliability_target: float = 0.90,
    week_min: float = 10.0,
    week_max: float = 25.0,
    week_step: float = 0.1,
) -> pd.DataFrame:
    if not 0 < reliability_target < 1:
        raise ValueError("reliability_target must lie between zero and one")
    if week_step <= 0 or week_min >= week_max:
        raise ValueError("Invalid gestational-week grid")
    weeks = np.arange(week_min, week_max + week_step / 2.0, week_step)
    records: list[dict[str, Any]] = []
    for row in profiles.itertuples(index=False):
        probabilities = predict_attainment_probability(
            fit,
            prepared_data,
            weeks,
            np.full_like(weeks, float(row.bmi)),
        )
        eligible = np.flatnonzero(probabilities >= reliability_target)
        reached = bool(eligible.size)
        index = int(eligible[0]) if reached else len(weeks) - 1
        records.append(
            {
                "mother_code": str(row.mother_code),
                "bmi": float(row.bmi),
                "recommended_week": float(np.round(weeks[index], 10)),
                "predicted_probability": float(probabilities[index]),
                "reliability_target": float(reliability_target),
                "reliability_reached_by_week_25": reached,
```

```
            }
        )
    return pd.DataFrame.from_records(records).sort_values(
        ["bmi", "mother_code"], kind="mergesort"
    ).reset_index(drop=True)


def estimate_y_measurement_error(male: pd.DataFrame) -> dict[str, float | int]:
    grouped = male.groupby("technical_event_id", sort=False)["y_concentration"]
    differences: list[float] = []
    for _, values in grouped:
        if len(values) == 2:
            pair = values.to_numpy(dtype=float)
            differences.append(float(pair[0] - pair[1]))
        elif len(values) > 2:
            raise ValueError("Measurement-error estimator expects at most two technical replicates")
    if not differences:
        raise ValueError("No technical replicate pairs are available")
    difference_array = np.asarray(differences, dtype=float)
    pair_count = len(difference_array)
    sum_of_squares = float(np.sum(difference_array**2))
    sigma = float(np.sqrt(sum_of_squares / (2.0 * pair_count)))
    lower = float(np.sqrt(sum_of_squares / (2.0 * chi2.ppf(0.975, pair_count))))
    upper = float(np.sqrt(sum_of_squares / (2.0 * chi2.ppf(0.025, pair_count))))
    return {
        "technical_pair_count": pair_count,
        "measurement_sigma": sigma,
        "lower_95": lower,
        "upper_95": upper,
        "median_absolute_pair_difference": float(np.median(np.abs(difference_array))),
        "maximum_absolute_pair_difference": float(np.max(np.abs(difference_array))),
    }


def optimal_contiguous_partition(
    individual: pd.DataFrame,
    *,
    n_groups: int,
    minimum_group_size: int,
    complexity_penalty_weeks: float = 0.0,
) -> Q2Policy:
    if n_groups < 1 or minimum_group_size < 1:
        raise ValueError("n_groups and minimum_group_size must be positive")
    ordered = individual.sort_values(["bmi", "mother_code"], kind="mergesort").reset_index(drop=True)
    n_mothers = len(ordered)
    if n_groups * minimum_group_size > n_mothers:
        raise ValueError("Minimum group-size constraint is infeasible")

    weeks = ordered["recommended_week"].to_numpy(dtype=float)
    bmis = ordered["bmi"].to_numpy(dtype=float)
    segment_cost = np.full((n_mothers, n_mothers), np.inf)
    segment_week = np.full((n_mothers, n_mothers), np.nan)
    prefix_week_sum = np.concatenate(([0.0], np.cumsum(weeks)))
    for start in range(n_mothers):
        running_max = -np.inf
        for end in range(start + minimum_group_size - 1, n_mothers):
            if end == start + minimum_group_size - 1:
                running_max = float(np.max(weeks[start : end + 1]))
            else:
                running_max = max(running_max, float(weeks[end]))
            common_week = running_max
            segment_week[start, end] = common_week
            segment_cost[start, end] = float(
                (end - start + 1) * common_week
                - (prefix_week_sum[end + 1] - prefix_week_sum[start])
            )

    dynamic = np.full((n_groups + 1, n_mothers + 1), np.inf)
    predecessor = np.full((n_groups + 1, n_mothers + 1), -1, dtype=int)
    dynamic[0, 0] = 0.0
    for group_count in range(1, n_groups + 1):
        earliest_end = group_count * minimum_group_size
        for end_exclusive in range(earliest_end, n_mothers + 1):
            earliest_start = (group_count - 1) * minimum_group_size
            latest_start = end_exclusive - minimum_group_size
            for start in range(earliest_start, latest_start + 1):
                candidate = dynamic[group_count - 1, start] + segment_cost[
                    start, end_exclusive - 1
                ]
                if candidate < dynamic[group_count, end_exclusive]:
                    dynamic[group_count, end_exclusive] = candidate
                    predecessor[group_count, end_exclusive] = start

    total_regret = float(dynamic[n_groups, n_mothers])
    if not np.isfinite(total_regret):
        raise RuntimeError("No feasible contiguous partition found")
    segments: list[tuple[int, int]] = []
    end_exclusive = n_mothers
    for group_count in range(n_groups, 0, -1):
        start = int(predecessor[group_count, end_exclusive])
        segments.append((start, end_exclusive - 1))
        end_exclusive = start
    segments.reverse()

    boundaries = [float((bmis[left_end] + bmis[left_end + 1]) / 2.0) for _, left_end in segments[:-1]]
    groups: list[dict[str, float | int | bool]] = []
    for index, (start, end) in enumerate(segments):
        lower = float(bmis[0]) if index == 0 else boundaries[index - 1]
        upper = float(bmis[-1]) if index == n_groups - 1 else boundaries[index]
        common_week = float(segment_week[start, end])
        subgroup = ordered.iloc[start : end + 1]
        groups.append(
            {
                "group": index + 1,
                "bmi_lower": lower,
                "bmi_upper": upper,
                "observed_bmi_min": float(bmis[start]),
                "observed_bmi_max": float(bmis[end]),
                "n_mothers": int(end - start + 1),
                "recommended_week": common_week,
```

```
                "mean_individual_week": float(subgroup["recommended_week"].mean()),
                "mean_excess_wait_weeks": float(
                    common_week - subgroup["recommended_week"].mean()
                ),
                "unreached_reliability_by_week_25": int(
                    (-subgroup["reliability_reached_by_week_25"]).sum()
                ),
            }
        )
    mean_regret = total_regret / n_mothers
    penalized = mean_regret + complexity_penalty_weeks * (n_groups - 1)
    return Q2Policy(
        n_groups=n_groups,
        groups=groups,
        total_regret_weeks=total_regret,
        mean_regret_weeks=mean_regret,
        complexity_penalty_weeks=float(complexity_penalty_weeks),
        penalized_objective=float(penalized),
    )


def select_group_count(
    individual: pd.DataFrame,
    *,
    candidate_group_counts: Iterable[int],
    minimum_group_size: int,
    complexity_penalty_weeks: float,
) -> tuple[Q2Policy, pd.DataFrame]:
    policies: list[Q2Policy] = []
    records: list[dict[str, float | int]] = []
    for n_groups in candidate_group_counts:
        policy = optimal_contiguous_partition(
            individual,
            n_groups=int(n_groups),
            minimum_group_size=minimum_group_size,
            complexity_penalty_weeks=complexity_penalty_weeks,
        )
        policies.append(policy)
        records.append(
            {
                "n_groups": policy.n_groups,
                "mean_regret_weeks": policy.mean_regret_weeks,
                "complexity_component": complexity_penalty_weeks * (policy.n_groups - 1),
                "penalized_objective": policy.penalized_objective,
            }
        )
    tradeoff = pd.DataFrame.from_records(records)
    selected_index = int(tradeoff["penalized_objective"].idxmin())
    return policies[selected_index], tradeoff


def analyze_q2(
    male: pd.DataFrame,
    events: pd.DataFrame,
    *,
    y_threshold: float = 0.04,
    reliability_target: float = 0.90,
    minimum_group_size: int = 30,
    complexity_penalty_weeks: float = 0.2,
) -> Q2Analysis:
    prepared = prepare_q2_data(events, y_threshold=y_threshold)
    model = fit_q2_attainment_model(prepared)
    profiles = baseline_mother_profiles(events)
    individual = individual_recommended_weeks(
        profiles,
        model.selected_fit,
        prepared,
        reliability_target=reliability_target,
    )
    policy, tradeoff = select_group_count(
        individual,
        candidate_group_counts=range(2, 7),
        minimum_group_size=minimum_group_size,
        complexity_penalty_weeks=complexity_penalty_weeks,
    )
    return Q2Analysis(
        prepared_data=prepared,
        model=model,
        profiles=profiles,
        individual=individual,
        policy=policy,
        group_count_tradeoff=tradeoff,
        measurement_error=estimate_y_measurement_error(male),
        reliability_target=reliability_target,
        y_threshold=y_threshold,
        minimum_group_size=minimum_group_size,
        complexity_penalty_weeks=complexity_penalty_weeks,
    )


def threshold_reliability_scenarios(
    events: pd.DataFrame,
    *,
    y_thresholds: Iterable[float],
    reliability_targets: Iterable[float],
    minimum_group_size: int,
    complexity_penalty_weeks: float,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    profiles = baseline_mother_profiles(events)
    summary_records: list[dict[str, float | int | str]] = []
    policy_records: list[dict[str, float | int | str | bool]] = []
    for threshold in y_thresholds:
        prepared = prepare_q2_data(events, y_threshold=float(threshold))
        model = fit_q2_attainment_model(prepared)
        for reliability in reliability_targets:
            individual = individual_recommended_weeks(
                profiles,
                model.selected_fit,
                prepared,
                reliability_target=float(reliability),
```

```
            )
            policy, _ = select_group_count(
                individual,
                candidate_group_counts=range(2, 7),
                minimum_group_size=minimum_group_size,
                complexity_penalty_weeks=complexity_penalty_weeks,
            )
            scenario_id = f"threshold_{float(threshold):.3f}_reliability_{float(reliability):.2f}"
            summary_records.append(
                {
                    "scenario_id": scenario_id,
                    "y_threshold": float(threshold),
                    "reliability_target": float(reliability),
                    "selected_group_count": policy.n_groups,
                    "mean_individual_week": float(individual["recommended_week"].mean()),
                    "unreached_by_week_25": int(
                        (~individual["reliability_reached_by_week_25"]).sum()
                    ),
                    "mean_regret_weeks": policy.mean_regret_weeks,
                    "penalized_objective": policy.penalized_objective,
                    "attainment_rate": float(prepared["attained"].mean()),
                    "week_coefficient": float(model.selected_fit.params["week_centered"]),
                    "bmi_coefficient": float(model.selected_fit.params["bmi_z"]),
                }
            )
            for group in policy.groups:
                policy_records.append({"scenario_id": scenario_id, **group})
    return pd.DataFrame.from_records(summary_records), pd.DataFrame.from_records(policy_records)


def _policy_resample_record(
    replicate: int,
    policy: Q2Policy,
    fit: Any,
    attained_rate: float,
) -> dict[str, float | int | str]:
    record: dict[str, float | int | str] = {
        "replicate": replicate,
        "status": "ok",
        "converged": bool(fit.converged),
        "attainment_rate": float(attained_rate),
        "week_coefficient": float(fit.params["week_centered"]),
        "bmi_coefficient": float(fit.params["bmi_z"]),
        "mean_regret_weeks": policy.mean_regret_weeks,
    }
    for index, group in enumerate(policy.groups, start=1):
        record[f"week_{index}"] = float(group["recommended_week"])
        record[f"n_mothers_{index}"] = int(group["n_mothers"])
        if index < policy.n_groups:
            record[f"cutpoint_{index}"] = float(group["bmi_upper"])
    return record


def simulate_measurement_error_policy(
    male: pd.DataFrame,
    events: pd.DataFrame,
    *,
    n_replicates: int,
    random_seed: int,
    n_groups: int = 4,
    minimum_group_size: int = 30,
    y_threshold: float = 0.04,
    reliability_target: float = 0.90,
) -> pd.DataFrame:
    if n_replicates < 1:
        raise ValueError("n_replicates must be positive")
    sigma = float(estimate_y_measurement_error(male)["measurement_sigma"])
    rng = np.random.default_rng(random_seed)
    profiles = baseline_mother_profiles(events)
    records: list[dict[str, float | int | str]] = []
    replicate_counts = events["technical_replicate_count"].to_numpy(dtype=float)
    observed_y = events["y_concentration"].to_numpy(dtype=float)
    for replicate in range(n_replicates):
        simulated = events.copy(deep=True)
        noise = rng.normal(0.0, sigma / np.sqrt(replicate_counts), size=len(events))
        simulated.loc[:, "y_concentration"] = np.clip(observed_y + noise, 1e-8, 1 - 1e-8)
        try:
            prepared = prepare_q2_data(simulated, y_threshold=y_threshold)
            selected_fit = _fit_q2_selected(prepared)
            if not selected_fit.converged:
                raise RuntimeError("NonConvergence")
            individual = individual_recommended_weeks(
                profiles,
                selected_fit,
                prepared,
                reliability_target=reliability_target,
            )
            policy = optimal_contiguous_partition(
                individual,
                n_groups=n_groups,
                minimum_group_size=minimum_group_size,
            )
            records.append(
                _policy_resample_record(
                    replicate,
                    policy,
                    selected_fit,
                    float(prepared["attained"].mean()),
                )
            )
        except Exception as exc:  # pragma: no cover - retained in formal audit output
            records.append(
                {
                    "replicate": replicate,
                    "status": f"failed:{type(exc).__name__}",
                    "converged": False,
                    "attainment_rate": float("nan"),
                }
            )
    return pd.DataFrame.from_records(records)
```

```
def bootstrap_q2_policy(
    events: pd.DataFrame,
    *,
    n_replicates: int,
    random_seed: int,
    n_groups: int = 4,
    minimum_group_size: int = 30,
    y_threshold: float = 0.04,
    reliability_target: float = 0.90,
) -> pd.DataFrame:
    if n_replicates < 1:
        raise ValueError("n_replicates must be positive")
    rng = np.random.default_rng(random_seed)
    mother_codes = events["mother_code"].astype(str).drop_duplicates().to_numpy()
    by_mother = {code: frame.copy(deep=True) for code, frame in events.groupby("mother_code")}
    records: list[dict[str, float | int | str]] = []
    for replicate in range(n_replicates):
        sampled = rng.choice(mother_codes, size=len(mother_codes), replace=True)
        pieces: list[pd.DataFrame] = []
        for draw_index, code in enumerate(sampled):
            piece = by_mother[code].copy(deep=True)
            clone_code = f"bootstrap_{replicate:04d}_{draw_index:04d}"
            piece.loc[:, "mother_code"] = clone_code
            piece.loc[:, "technical_event_id"] = (
                clone_code + "|" + piece["technical_event_id"].astype(str)
            )
            pieces.append(piece)
        bootstrap_events = pd.concat(pieces, ignore_index=True)
        try:
            prepared = prepare_q2_data(bootstrap_events, y_threshold=y_threshold)
            selected_fit = _fit_q2_selected(prepared)
            if not selected_fit.converged:
                raise RuntimeError("NonConvergence")
            profiles = baseline_mother_profiles(bootstrap_events)
            individual = individual_recommended_weeks(
                profiles,
                selected_fit,
                prepared,
                reliability_target=reliability_target,
            )
            policy = optimal_contiguous_partition(
                individual,
                n_groups=n_groups,
                minimum_group_size=minimum_group_size,
            )
            records.append(
                _policy_resample_record(
                    replicate,
                    policy,
                    selected_fit,
                    float(prepared["attained"].mean()),
                )
            )
        except Exception as exc:  # pragma: no cover - retained in formal audit output
            records.append(
                {
                    "replicate": replicate,
                    "status": f"failed:{type(exc).__name__}",
                    "converged": False,
                    "attainment_rate": float("nan"),
                }
            )
    return pd.DataFrame.from_records(records)


def _coefficient_table(fit: Any) -> pd.DataFrame:
    estimates = fit.params.astype(float)
    standard_errors = fit.bse.astype(float)
    return pd.DataFrame(
        {
            "term": list(estimates.index),
            "estimate": estimates.to_numpy(),
            "robust_standard_error": standard_errors.to_numpy(),
            "lower_95": (estimates - 1.96 * standard_errors).to_numpy(),
            "upper_95": (estimates + 1.96 * standard_errors).to_numpy(),
            "p_value": fit.pvalues.astype(float).to_numpy(),
        }
    )


def _save_figure(fig: plt.Figure, output_dir: Path, stem: str) -> None:
    fig.savefig(
        output_dir / f"{stem}.png",
        dpi=300,
        bbox_inches="tight",
        facecolor="white",
        metadata={"Software": "CUMCM2025-C reproducible pipeline"},
    )
    fig.savefig(
        output_dir / f"{stem}.pdf",
        bbox_inches="tight",
        facecolor="white",
        metadata={"Creator": "CUMCM2025-C reproducible pipeline"},
    )
    plt.close(fig)


def _plot_probability_surface(analysis: Q2Analysis, output_dir: Path) -> None:
    weeks = np.linspace(10, 25, 151)
    bmis = np.linspace(
        float(analysis.profiles["bmi"].min()),
        float(analysis.profiles["bmi"].max()),
        151,
    )
    week_grid, bmi_grid = np.meshgrid(weeks, bmis)
    probability = predict_attainment_probability(
        analysis.model.selected_fit,
        analysis.prepared_data,
```

```
        week_grid,
        bmi_grid,
    )
    fig, axis = plt.subplots(figsize=(7.4, 4.9))
    filled = axis.contourf(
        week_grid,
        bmi_grid,
        probability,
        levels=np.linspace(0.0, 1.0, 21),
        cmap="viridis",
    )
    contour = axis.contour(
        week_grid,
        bmi_grid,
        probability,
        levels=[analysis.reliability_target],
        colors=["#C62828"],
        linewidths=2.0,
    )
    axis.clabel(contour, fmt={analysis.reliability_target: f"{analysis.reliability_target:.0%}"})
    axis.set(xlabel="Gestational week", ylabel="BMI")
    colorbar = fig.colorbar(filled, ax=axis, pad=0.02)
    colorbar.set_label("Predicted probability of Y >= 4%")
    fig.tight_layout()
    _save_figure(fig, output_dir, "q2_probability_surface")


def _plot_policy(analysis: Q2Analysis, output_dir: Path) -> None:
    fig, axis = plt.subplots(figsize=(7.4, 4.9))
    axis.scatter(
        analysis.individual["bmi"],
        analysis.individual["recommended_week"],
        s=18,
        alpha=0.42,
        color="#4472A6",
        linewidths=0,
        label="Individual earliest reliable week",
    )
    for group in analysis.policy.groups:
        axis.hlines(
            float(group["recommended_week"]),
            float(group["bmi_lower"]),
            float(group["bmi_upper"]),
            colors="#C43E32",
            linewidth=3.0,
        )
    for group in analysis.policy.groups[:-1]:
        axis.axvline(float(group["bmi_upper"]), color="#666666", linestyle="--", linewidth=0.9)
    axis.set(xlabel="Baseline BMI", ylabel="Recommended gestational week")
    axis.set_ylim(10, 25.5)
    axis.grid(alpha=0.2)
    axis.legend(frameon=False)
    fig.tight_layout()
    _save_figure(fig, output_dir, "q2_policy")


def _plot_sensitivity(summary: pd.DataFrame, output_dir: Path) -> None:
    pivot = summary.pivot(
        index="y_threshold",
        columns="reliability_target",
        values="mean_individual_week",
    ).sort_index().sort_index(axis=1)
    fig, axis = plt.subplots(figsize=(6.4, 4.6))
    image = axis.imshow(pivot.to_numpy(), cmap="YlOrRd", aspect="auto")
    axis.set_xticks(range(len(pivot.columns)), [f"{value:.0%}" for value in pivot.columns])
    axis.set_yticks(range(len(pivot.index)), [f"{value:.1%}" for value in pivot.index])
    axis.set(xlabel="Reliability target", ylabel="Y threshold")
    for row in range(len(pivot.index)):
        for column in range(len(pivot.columns)):
            axis.text(column, row, f"{pivot.iloc[row, column]:.1f}", ha="center", va="center")
    colorbar = fig.colorbar(image, ax=axis, pad=0.02)
    colorbar.set_label("Mean individual recommended week")
    fig.tight_layout()
    _save_figure(fig, output_dir, "q2_sensitivity")


def _sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def write_q2_outputs(
    analysis: Q2Analysis,
    measurement_simulation: pd.DataFrame,
    bootstrap: pd.DataFrame,
    output_dir: str | Path,
) -> dict[str, Any]:
    destination = Path(output_dir)
    destination.mkdir(parents=True, exist_ok=True)
    scenario_summary, scenario_policies = threshold_reliability_scenarios(
        analysis.prepared_data,
        y_thresholds=(0.035, 0.04, 0.045),
        reliability_targets=(0.85, 0.90, 0.95),
        minimum_group_size=analysis.minimum_group_size,
        complexity_penalty_weeks=analysis.complexity_penalty_weeks,
    )

    metrics_payload = {
        "schema_version": "1.0",
        "selected_group_count": analysis.policy.n_groups,
        "y_threshold": analysis.y_threshold,
        "reliability_target": analysis.reliability_target,
        "minimum_group_size": analysis.minimum_group_size,
        "complexity_penalty_weeks": analysis.complexity_penalty_weeks,
        "attainment_model": {
            "parameters": {
                key: float(value) for key, value in analysis.model.selected_fit.params.items()
```

```
            },
            "p_values": {
                key: float(value) for key, value in analysis.model.selected_fit.pvalues.items()
            },
            "exchangeable_correlation": float(
                analysis.model.selected_fit.cov_struct.dep_params
            ),
            "interaction_p_value": analysis.model.interaction_p_value,
        },
        "measurement_error": analysis.measurement_error,
        "policy": {
            "mean_regret_weeks": analysis.policy.mean_regret_weeks,
            "penalized_objective": analysis.policy.penalized_objective,
            "groups": analysis.policy.groups,
        },
        "measurement_error_replicates": int(len(measurement_simulation)),
        "bootstrap_replicates": int(len(bootstrap)),
        "measurement_error_failures": int((measurement_simulation["status"] != "ok").sum()),
        "bootstrap_failures": int((bootstrap["status"] != "ok").sum()),
    }
    (destination / "q2_metrics.json").write_text(
        json.dumps(metrics_payload, ensure_ascii=False, sort_keys=True, indent=2) + "\n",
        encoding="utf-8",
        newline="\n",
    )
    _coefficient_table(analysis.model.selected_fit).to_csv(
        destination / "q2_model_coefficients.csv", index=False, lineterminator="\n"
    )
    analysis.individual.to_csv(
        destination / "q2_individual_weeks.csv", index=False, lineterminator="\n"
    )
    pd.DataFrame(analysis.policy.groups).to_csv(
        destination / "q2_group_policy.csv", index=False, lineterminator="\n"
    )
    analysis.group_count_tradeoff.to_csv(
        destination / "q2_group_count_tradeoff.csv", index=False, lineterminator="\n"
    )
    scenario_summary.to_csv(
        destination / "q2_scenario_summary.csv", index=False, lineterminator="\n"
    )
    scenario_policies.to_csv(
        destination / "q2_scenario_policies.csv", index=False, lineterminator="\n"
    )
    measurement_simulation.to_csv(
        destination / "q2_measurement_error_simulation.csv", index=False, lineterminator="\n"
    )
    bootstrap.to_csv(destination / "q2_bootstrap.csv", index=False, lineterminator="\n")

    _plot_probability_surface(analysis, destination)
    _plot_policy(analysis, destination)
    _plot_sensitivity(scenario_summary, destination)

    artifacts: dict[str, dict[str, int | str]] = {}
    for path in sorted(destination.iterdir()):
        if path.is_file() and path.name != "q2_manifest.json":
            artifacts[path.name] = {"sha256": _sha256(path), "bytes": path.stat().st_size}
    manifest = {"schema_version": "1.0", "artifacts": artifacts}
    (destination / "q2_manifest.json").write_text(
        json.dumps(manifest, ensure_ascii=False, sort_keys=True, indent=2) + "\n",
        encoding="utf-8",
        newline="\n",
    )
    return manifest
```

04_code/cumcm2025c/q3_multifactor.py

```
from __future__ import annotations

import hashlib
import json
from dataclasses import dataclass
from pathlib import Path
from typing import Any

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.special import expit, logit
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    average_precision_score,
    brier_score_loss,
    log_loss,
    roc_auc_score,
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

from .model_data import stratified_mother_splits
from .q2_timing import (
    Q2Policy,
    baseline_mother_profiles,
    select_group_count,
)


PRETEST_FEATURES = (
    "gestational_week",
    "bmi",
    "age",
    "height_cm",
```

```
    "weight_kg",
    "ivf_assisted",
    "pregnancies_numeric",
    "births",
)
BASELINE_FEATURES = ("gestational_week", "bmi")
MODEL_NAMES = ("bmi_baseline", "maternal_elastic_net", "maternal_monotone_boosting")


@dataclass
class Q3Evaluation:
    oof_predictions: pd.DataFrame
    metrics: pd.DataFrame
    fold_audit: pd.DataFrame
    tuning: pd.DataFrame
    selected_model: str


@dataclass
class CalibratedPretestModel:
    model_name: str
    features: tuple[str, ...]
    pipeline: Pipeline
    calibration_intercept: float
    calibration_slope: float
    parameters: dict[str, Any]

    def predict_probability(self, frame: pd.DataFrame) -> np.ndarray:
        raw = self.pipeline.predict_proba(frame.loc[:, self.features])[:, 1]
        clipped = np.clip(raw, 1e-6, 1 - 1e-6)
        return expit(self.calibration_intercept + self.calibration_slope * logit(clipped))


@dataclass
class Q3Analysis:
    data: pd.DataFrame
    evaluation: Q3Evaluation
    final_model: CalibratedPretestModel
    profiles: pd.DataFrame
    individual: pd.DataFrame
    policy: Q2Policy
    group_count_tradeoff: pd.DataFrame
    reliability_target: float


def prepare_q3_data(events: pd.DataFrame) -> pd.DataFrame:
    data = events.copy(deep=True)
    data = data.assign(target=data["y_attained"].astype(int))
    required = [*PRETEST_FEATURES, "target", "mother_code"]
    missing = [column for column in required if column not in data]
    if missing:
        raise ValueError(f"Q3 data are missing required fields: {missing}")
    if data.loc[:, required].isna().any().any():
        counts = data.loc[:, required].isna().sum().loc[lambda values: values.gt(0)]
        raise ValueError(f"Q3 data contain missing fields: {counts.to_dict()}")
    return data


def _candidate_parameters(model_name: str) -> list[dict[str, Any]]:
    if model_name == "bmi_baseline":
        return [{"C": 10.0}]
    if model_name == "maternal_elastic_net":
        return [{"C": value, "l1_ratio": 0.5} for value in (0.05, 0.2, 1.0, 5.0)]
    if model_name == "maternal_monotone_boosting":
        return [
            {
                "max_leaf_nodes": leaves,
                "min_samples_leaf": minimum_leaf,
                "l2_regularization": 5.0,
                "learning_rate": 0.05,
                "max_iter": 250,
            }
            for leaves in (7, 15)
            for minimum_leaf in (20, 40)
        ]
    raise ValueError(f"Unknown Q3 model: {model_name}")


def _features_for(model_name: str) -> tuple[str, ...]:
    return BASELINE_FEATURES if model_name == "bmi_baseline" else PRETEST_FEATURES


def _build_pipeline(model_name: str, parameters: dict[str, Any], random_seed: int) -> Pipeline:
    if model_name == "bmi_baseline":
        classifier = LogisticRegression(
            C=float(parameters["C"]),
            solver="lbfgs",
            max_iter=5000,
            random_state=random_seed,
        )
        return Pipeline(
            [
                ("impute", SimpleImputer(strategy="median")),
                ("scale", StandardScaler()),
                ("model", classifier),
            ]
        )
    if model_name == "maternal_elastic_net":
        classifier = LogisticRegression(
            C=float(parameters["C"]),
            l1_ratio=float(parameters["l1_ratio"]),
            solver="saga",
            max_iter=5000,
            random_state=random_seed,
        )
        return Pipeline(
            [
                ("impute", SimpleImputer(strategy="median")),
                ("scale", StandardScaler()),
                ("model", classifier),
```

```
            ]
        )
    if model_name == "maternal_monotone_boosting":
        monotonic_constraints = [1, -1, 0, 0, 0, 0, 0]
        classifier = HistGradientBoostingClassifier(
            **parameters,
            monotonic_cst=monotonic_constraints,
            early_stopping=False,
            random_state=random_seed,
        )
        return Pipeline(
            [
                ("impute", SimpleImputer(strategy="median")),
                ("model", classifier),
            ]
        )
    raise ValueError(f"Unknown Q3 model: {model_name}")


def _inner_oof_probabilities(
    data: pd.DataFrame,
    model_name: str,
    parameters: dict[str, Any],
    *,
    n_splits: int,
    random_seed: int,
) -> np.ndarray:
    y = data["target"].to_numpy(dtype=int)
    groups = data["mother_code"].astype(str).to_numpy()
    probabilities = np.full(len(data), np.nan)
    features = _features_for(model_name)
    for fold, (train_idx, test_idx) in enumerate(
        stratified_mother_splits(y, groups, n_splits=n_splits, random_state=random_seed)
    ):
        pipeline = _build_pipeline(model_name, parameters, random_seed + fold)
        pipeline.fit(data.iloc[train_idx].loc[:, features], y[train_idx])
        probabilities[test_idx] = pipeline.predict_proba(
            data.iloc[test_idx].loc[:, features]
        )[:, 1]
    if np.isnan(probabilities).any():
        raise RuntimeError("Inner grouped CV did not predict every row")
    return probabilities


def _tune_model(
    data: pd.DataFrame,
    model_name: str,
    *,
    inner_folds: int,
    random_seed: int,
) -> tuple[dict[str, Any], np.ndarray, float]:
    y = data["target"].to_numpy(dtype=int)
    best_parameters: dict[str, Any] | None = None
    best_probabilities: np.ndarray | None = None
    best_brier = np.inf
    for parameters in _candidate_parameters(model_name):
        probabilities = _inner_oof_probabilities(
            data,
            model_name,
            parameters,
            n_splits=inner_folds,
            random_seed=random_seed,
        )
        score = float(brier_score_loss(y, probabilities))
        if score < best_brier - 1e-12:
            best_brier = score
            best_parameters = parameters
            best_probabilities = probabilities
    if best_parameters is None or best_probabilities is None:
        raise RuntimeError("Q3 tuning produced no candidate")
    return best_parameters, best_probabilities, best_brier


def _fit_platt(probabilities: np.ndarray, target: np.ndarray) -> tuple[float, float]:
    transformed = logit(np.clip(probabilities, 1e-6, 1 - 1e-6)).reshape(-1, 1)
    calibrator = LogisticRegression(C=1e6, solver="lbfgs", max_iter=5000)
    calibrator.fit(transformed, target)
    intercept = float(calibrator.intercept_[0])
    slope = float(calibrator.coef_[0, 0])
    if slope <= 0:
        raise RuntimeError("Platt calibration slope is not positive")
    return intercept, slope


def _apply_platt(probabilities: np.ndarray, intercept: float, slope: float) -> np.ndarray:
    return expit(intercept + slope * logit(np.clip(probabilities, 1e-6, 1 - 1e-6)))


def _expected_calibration_error(target: np.ndarray, probabilities: np.ndarray, bins: int = 10) -> float:
    order = np.argsort(probabilities)
    chunks = np.array_split(order, bins)
    error = 0.0
    for chunk in chunks:
        if len(chunk) == 0:
            continue
        error += len(chunk) / len(target) * abs(
            float(np.mean(target[chunk])) - float(np.mean(probabilities[chunk]))
        )
    return float(error)


def _calibration_intercept_slope(target: np.ndarray, probabilities: np.ndarray) -> tuple[float, float]:
    return _fit_platt(probabilities, target)


def _metric_row(model_name: str, target: np.ndarray, probabilities: np.ndarray) -> dict[str, float | str]:
    calibration_intercept, calibration_slope = _calibration_intercept_slope(
        target, probabilities
    )
    return {
```

```
        "model": model_name,
        "roc_auc": float(roc_auc_score(target, probabilities)),
        "pr_auc": float(average_precision_score(target, probabilities)),
        "brier": float(brier_score_loss(target, probabilities)),
        "log_loss": float(log_loss(target, probabilities)),
        "ece_10": _expected_calibration_error(target, probabilities, bins=10),
        "calibration_intercept": calibration_intercept,
        "calibration_slope": calibration_slope,
    }


def evaluate_q3_models(
    data: pd.DataFrame,
    *,
    outer_folds: int,
    inner_folds: int,
    random_seed: int,
) -> Q3Evaluation:
    y = data["target"].to_numpy(dtype=int)
    groups = data["mother_code"].astype(str).to_numpy()
    outer_splits = list(
        stratified_mother_splits(
            y,
            groups,
            n_splits=outer_folds,
            random_state=random_seed,
        )
    )
    predictions = {
        model_name: np.full(len(data), np.nan, dtype=float) for model_name in MODEL_NAMES
    }
    outer_fold_column = np.full(len(data), -1, dtype=int)
    audit_records: list[dict[str, int]] = []
    tuning_records: list[dict[str, Any]] = []

    for outer_fold, (train_idx, test_idx) in enumerate(outer_splits):
        outer_fold_column[test_idx] = outer_fold
        train_groups = set(groups[train_idx])
        test_groups = set(groups[test_idx])
        audit_records.append(
            {
                "outer_fold": outer_fold,
                "train_rows": len(train_idx),
                "test_rows": len(test_idx),
                "train_mothers": len(train_groups),
                "test_mothers": len(test_groups),
                "mother_overlap": len(train_groups.intersection(test_groups)),
            }
        )
        train_data = data.iloc[train_idx].reset_index(drop=True)
        for model_position, model_name in enumerate(MODEL_NAMES):
            parameters, inner_probabilities, inner_brier = _tune_model(
                train_data,
                model_name,
                inner_folds=inner_folds,
                random_seed=random_seed + 100 * outer_fold + 10 * model_position,
            )
            calibration_intercept, calibration_slope = _fit_platt(
                inner_probabilities,
                train_data["target"].to_numpy(dtype=int),
            )
            pipeline = _build_pipeline(
                model_name,
                parameters,
                random_seed + 1000 + outer_fold,
            )
            features = _features_for(model_name)
            pipeline.fit(data.iloc[train_idx].loc[:, features], y[train_idx])
            raw_test = pipeline.predict_proba(data.iloc[test_idx].loc[:, features])[:, 1]
            predictions[model_name][test_idx] = _apply_platt(
                raw_test,
                calibration_intercept,
                calibration_slope,
            )
            tuning_records.append(
                {
                    "outer_fold": outer_fold,
                    "model": model_name,
                    "parameters": json.dumps(parameters, sort_keys=True),
                    "inner_brier": inner_brier,
                    "calibration_intercept": calibration_intercept,
                    "calibration_slope": calibration_slope,
                }
            )

    if (outer_fold_column < 0).any() or any(np.isnan(values).any() for values in predictions.values()):
        raise RuntimeError("Outer grouped CV did not predict every row exactly once")
    oof = pd.DataFrame(
        {
            "row_index": np.arange(len(data)),
            "mother_code": groups,
            "target": y,
            "outer_fold": outer_fold_column,
            **{f"prob_{name}": values for name, values in predictions.items()},
        }
    )
    metrics = pd.DataFrame.from_records(
        [_metric_row(name, y, predictions[name]) for name in MODEL_NAMES]
    )
    maternal = metrics[metrics["model"].ne("bmi_baseline")]
    best_maternal_index = int(maternal["brier"].idxmin())
    best_maternal = str(metrics.loc[best_maternal_index, "model"])
    baseline_brier = float(metrics.loc[metrics["model"].eq("bmi_baseline"), "brier"].iloc[0])
    selected = (
        best_maternal
        if float(metrics.loc[best_maternal_index, "brier"]) <= baseline_brier
        else "bmi_baseline"
    )
    return Q3Evaluation(
        oof_predictions=oof,
```

```
        metrics=metrics,
        fold_audit=pd.DataFrame.from_records(audit_records),
        tuning=pd.DataFrame.from_records(tuning_records),
        selected_model=selected,
    )


def _fit_final_calibrated_model(
    data: pd.DataFrame,
    model_name: str,
    *,
    random_seed: int,
) -> CalibratedPretestModel:
    parameters, oof_raw, _ = _tune_model(
        data,
        model_name,
        inner_folds=5,
        random_seed=random_seed,
    )
    calibration_intercept, calibration_slope = _fit_platt(
        oof_raw,
        data["target"].to_numpy(dtype=int),
    )
    pipeline = _build_pipeline(model_name, parameters, random_seed + 999)
    features = _features_for(model_name)
    pipeline.fit(data.loc[:, features], data["target"].to_numpy(dtype=int))
    return CalibratedPretestModel(
        model_name=model_name,
        features=features,
        pipeline=pipeline,
        calibration_intercept=calibration_intercept,
        calibration_slope=calibration_slope,
        parameters=parameters,
    )


def q3_individual_recommended_weeks(
    profiles: pd.DataFrame,
    model: CalibratedPretestModel,
    *,
    reliability_target: float,
    week_min: float = 10.0,
    week_max: float = 25.0,
    week_step: float = 0.1,
) -> pd.DataFrame:
    weeks = np.arange(week_min, week_max + week_step / 2.0, week_step)
    records: list[dict[str, float | str | bool]] = []
    for _, profile in profiles.iterrows():
        repeated = pd.DataFrame([profile.to_dict()] * len(weeks))
        repeated.loc[:, "gestational_week"] = weeks
        probabilities = model.predict_probability(repeated)
        eligible = np.flatnonzero(probabilities >= reliability_target)
        reached = bool(eligible.size)
        index = int(eligible[0]) if reached else len(weeks) - 1
        records.append(
            {
                "mother_code": str(profile["mother_code"]),
                "bmi": float(profile["bmi"]),
                "recommended_week": float(np.round(weeks[index], 10)),
                "predicted_probability": float(probabilities[index]),
                "reliability_target": float(reliability_target),
                "reliability_reached_by_week_25": reached,
            }
        )
    return pd.DataFrame.from_records(records).sort_values(
        ["bmi", "mother_code"], kind="mergesort"
    ).reset_index(drop=True)


def analyze_q3(
    events: pd.DataFrame,
    *,
    evaluation: Q3Evaluation | None = None,
    reliability_target: float = 0.90,
    minimum_group_size: int = 30,
    complexity_penalty_weeks: float = 0.2,
    random_seed: int = 2025,
) -> Q3Analysis:
    data = prepare_q3_data(events)
    if evaluation is None:
        evaluation = evaluate_q3_models(
            data,
            outer_folds=5,
            inner_folds=4,
            random_seed=random_seed,
        )
    final_model = _fit_final_calibrated_model(
        data,
        evaluation.selected_model,
        random_seed=random_seed,
    )
    profiles = baseline_mother_profiles(events)
    individual = q3_individual_recommended_weeks(
        profiles,
        final_model,
        reliability_target=reliability_target,
    )
    policy, tradeoff = select_group_count(
        individual,
        candidate_group_counts=range(2, 7),
        minimum_group_size=minimum_group_size,
        complexity_penalty_weeks=complexity_penalty_weeks,
    )
    return Q3Analysis(
        data=data,
        evaluation=evaluation,
        final_model=final_model,
        profiles=profiles,
        individual=individual,
        policy=policy,
```

```
        group_count_tradeoff=tradeoff,
        reliability_target=reliability_target,
    )


def _calibration_table(evaluation: Q3Evaluation, bins: int = 10) -> pd.DataFrame:
    records: list[dict[str, float | int | str]] = []
    target = evaluation.oof_predictions["target"].to_numpy(dtype=int)
    for model_name in MODEL_NAMES:
        probabilities = evaluation.oof_predictions[f"prob_{model_name}"].to_numpy(dtype=float)
        for bin_index, indices in enumerate(np.array_split(np.argsort(probabilities), bins), start=1):
            if len(indices) == 0:
                continue
            records.append(
                {
                    "model": model_name,
                    "bin": bin_index,
                    "n": len(indices),
                    "mean_probability": float(np.mean(probabilities[indices])),
                    "observed_fraction": float(np.mean(target[indices])),
                }
            )
    return pd.DataFrame.from_records(records)


def _policy_regret_table(analysis: Q3Analysis) -> pd.DataFrame:
    ordered = analysis.individual.sort_values(
        ["bmi", "mother_code"], kind="mergesort"
    ).reset_index(drop=True)
    parts: list[pd.DataFrame] = []
    start = 0
    for group in analysis.policy.groups:
        count = int(group["n_mothers"])
        part = ordered.iloc[start : start + count].copy()
        part["group"] = int(group["group"])
        part["group_recommended_week"] = float(group["recommended_week"])
        part["excess_wait_weeks"] = (
            part["group_recommended_week"] - part["recommended_week"]
        )
        parts.append(part)
        start += count
    if start != len(ordered):
        raise RuntimeError("Q3 policy groups do not cover all mother profiles")
    return pd.concat(parts, ignore_index=True)


def _q3_sensitivity(analysis: Q3Analysis) -> tuple[pd.DataFrame, pd.DataFrame]:
    summaries: list[dict[str, float | int]] = []
    policies: list[dict[str, float | int]] = []
    for reliability_target in (0.85, 0.90, 0.95):
        for bmi_shift in (-1.0, 0.0, 1.0):
            profiles = analysis.profiles.copy(deep=True)
            profiles["bmi"] = profiles["bmi"] + bmi_shift
            individual = q3_individual_recommended_weeks(
                profiles,
                analysis.final_model,
                reliability_target=reliability_target,
            )
            policy, _ = select_group_count(
                individual,
                candidate_group_counts=range(2, 7),
                minimum_group_size=30,
                complexity_penalty_weeks=0.2,
            )
            scenario_id = f"r{reliability_target:.2f}_bmi{bmi_shift:+.1f}"
            summaries.append(
                {
                    "scenario_id": scenario_id,
                    "reliability_target": reliability_target,
                    "bmi_shift": bmi_shift,
                    "selected_group_count": policy.n_groups,
                    "mean_individual_week": float(individual["recommended_week"].mean()),
                    "unreached_by_week_25": int(
                        (-individual["reliability_reached_by_week_25"]).sum()
                    ),
                    "mean_regret_weeks": policy.mean_regret_weeks,
                    "penalized_objective": policy.penalized_objective,
                }
            )
            for group in policy.groups:
                policies.append({"scenario_id": scenario_id, **group})
    return pd.DataFrame.from_records(summaries), pd.DataFrame.from_records(policies)


def _q3_feature_effects(analysis: Q3Analysis) -> pd.DataFrame:
    estimator = analysis.final_model.pipeline.named_steps["model"]
    if hasattr(estimator, "coef_"):
        values = np.asarray(estimator.coef_, dtype=float).reshape(-1)
        return pd.DataFrame(
            {
                "feature": analysis.final_model.features,
                "effect_type": "standardized_log_odds_coefficient",
                "effect": values,
                "absolute_effect": np.abs(values),
            }
        ).sort_values("absolute_effect", ascending=False, kind="mergesort")

    frame = analysis.data.loc[:, analysis.final_model.features].astype(float).copy()
    records: list[dict[str, float | str]] = []
    for feature in analysis.final_model.features:
        low = float(frame[feature].quantile(0.25))
        high = float(frame[feature].quantile(0.75))
        low_frame = frame.copy()
        high_frame = frame.copy()
        low_frame[feature] = low
        high_frame[feature] = high
        difference = float(
            np.mean(analysis.final_model.predict_probability(high_frame))
            - np.mean(analysis.final_model.predict_probability(low_frame))
        )
```

```
        records.append(
            {
                "feature": feature,
                "effect_type": "mean_probability_iqr_contrast",
                "effect": difference,
                "absolute_effect": abs(difference),
            }
        )
    return pd.DataFrame.from_records(records).sort_values(
        "absolute_effect", ascending=False, kind="mergesort"
    )


def _save_figure(fig: plt.Figure, output_dir: Path, stem: str) -> None:
    fig.savefig(output_dir / f"{stem}.pdf", bbox_inches="tight")
    fig.savefig(output_dir / f"{stem}.png", dpi=220, bbox_inches="tight")
    plt.close(fig)


def _plot_q3_cv(metrics: pd.DataFrame, output_dir: Path) -> None:
    ordered = metrics.set_index("model").loc[list(MODEL_NAMES)].reset_index()
    x = np.arange(len(ordered))
    fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.2))
    axes[0].bar(x - 0.18, ordered["roc_auc"], width=0.36, label="ROC-AUC")
    axes[0].bar(x + 0.18, ordered["pr_auc"], width=0.36, label="PR-AUC")
    axes[0].set_ylim(0.5, 1.0)
    axes[0].set_xticks(x, ordered["model"], rotation=18, ha="right")
    axes[0].set_ylabel("Grouped out-of-fold score")
    axes[0].legend(frameon=False)
    axes[1].bar(x, ordered["brier"], color="#4C78A8")
    axes[1].set_xticks(x, ordered["model"], rotation=18, ha="right")
    axes[1].set_ylabel("Brier score (lower is better)")
    fig.tight_layout()
    _save_figure(fig, output_dir, "q3_cv_comparison")


def _plot_q3_calibration(calibration: pd.DataFrame, output_dir: Path) -> None:
    fig, ax = plt.subplots(figsize=(5.6, 5.0))
    ax.plot([0, 1], [0, 1], "--", color="0.55", label="Ideal")
    for model_name, group in calibration.groupby("model", sort=False):
        ax.plot(
            group["mean_probability"],
            group["observed_fraction"],
            marker="o",
            label=model_name,
        )
    ax.set(xlim=(0, 1), ylim=(0, 1), xlabel="Predicted probability", ylabel="Observed fraction")
    ax.legend(frameon=False, fontsize=8)
    fig.tight_layout()
    _save_figure(fig, output_dir, "q3_calibration")


def _plot_q3_regret(regret: pd.DataFrame, output_dir: Path) -> None:
    summary = regret.groupby("group", sort=True).agg(
        mean_excess_wait=("excess_wait_weeks", "mean"),
        maximum_excess_wait=("excess_wait_weeks", "max"),
    )
    x = np.arange(len(summary))
    fig, ax = plt.subplots(figsize=(6.4, 4.2))
    ax.bar(x - 0.18, summary["mean_excess_wait"], width=0.36, label="Mean")
    ax.bar(x + 0.18, summary["maximum_excess_wait"], width=0.36, label="Maximum")
    ax.set_xticks(x, [f"Group {int(value)}" for value in summary.index])
    ax.set_ylabel("Compression regret (weeks)")
    ax.legend(frameon=False)
    fig.tight_layout()
    _save_figure(fig, output_dir, "q3_policy_regret")


def _sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def write_q3_outputs(analysis: Q3Analysis, output_dir: str | Path) -> dict[str, Any]:
    destination = Path(output_dir)
    destination.mkdir(parents=True, exist_ok=True)
    calibration = _calibration_table(analysis.evaluation)
    regret = _policy_regret_table(analysis)
    sensitivity, sensitivity_policies = _q3_sensitivity(analysis)
    feature_effects = _q3_feature_effects(analysis)

    selected_metrics = analysis.evaluation.metrics.set_index("model").loc[
        analysis.evaluation.selected_model
    ]
    metrics_payload = {
        "schema_version": "1.0",
        "selected_model": analysis.evaluation.selected_model,
        "selected_parameters": analysis.final_model.parameters,
        "features": list(analysis.final_model.features),
        "reliability_target": analysis.reliability_target,
        "selected_group_count": analysis.policy.n_groups,
        "mean_policy_regret_weeks": analysis.policy.mean_regret_weeks,
        "mother_overlap_max": int(analysis.evaluation.fold_audit["mother_overlap"].max()),
        "all_outer_rows_predicted_once": bool(
            analysis.evaluation.oof_predictions["row_index"].nunique() == len(analysis.data)
        ),
        "selected_model_metrics": {
            key: float(value) for key, value in selected_metrics.items() if key != "model"
        },
    }
    (destination / "q3_metrics.json").write_text(
        json.dumps(metrics_payload, ensure_ascii=False, sort_keys=True, indent=2) + "\n",
        encoding="utf-8",
        newline="\n",
    )
    analysis.evaluation.metrics.to_csv(destination / "q3_cv_metrics.csv", index=False, lineterminator="\n")
    analysis.evaluation.oof_predictions.to_csv(destination / "q3_oof_predictions.csv", index=False, lineterminator="\n")
```

```
    analysis.evaluation.fold_audit.to_csv(destination / "q3_fold_audit.csv", index=False, lineterminator="\n")
    analysis.evaluation.tuning.to_csv(destination / "q3_tuning.csv", index=False, lineterminator="\n")
    analysis.individual.to_csv(destination / "q3_individual_weeks.csv", index=False, lineterminator="\n")
    pd.DataFrame(analysis.policy.groups).to_csv(destination / "q3_group_policy.csv", index=False, lineterminator="\n")
    analysis.group_count_tradeoff.to_csv(destination / "q3_group_count_tradeoff.csv", index=False, lineterminator="\n")
    regret.to_csv(destination / "q3_policy_regret.csv", index=False, lineterminator="\n")
    sensitivity.to_csv(destination / "q3_sensitivity.csv", index=False, lineterminator="\n")
    sensitivity_policies.to_csv(destination / "q3_sensitivity_policies.csv", index=False, lineterminator="\n")
    calibration.to_csv(destination / "q3_calibration.csv", index=False, lineterminator="\n")
    feature_effects.to_csv(destination / "q3_feature_effects.csv", index=False, lineterminator="\n")

    _plot_q3_cv(analysis.evaluation.metrics, destination)
    _plot_q3_calibration(calibration, destination)
    _plot_q3_regret(regret, destination)

    artifacts: dict[str, dict[str, int | str]] = {}
    for path in sorted(destination.iterdir()):
        if path.is_file() and path.name != "q3_manifest.json":
            artifacts[path.name] = {"sha256": _sha256(path), "bytes": path.stat().st_size}
    manifest = {
        "schema_version": "1.0",
        "selected_model": analysis.evaluation.selected_model,
        "artifacts": artifacts,
    }
    (destination / "q3_manifest.json").write_text(
        json.dumps(manifest, ensure_ascii=False, sort_keys=True, indent=2) + "\n",
        encoding="utf-8",
        newline="\n",
    )
    return manifest
```

04_code/cumcm2025c/q4_classifier.py

```
from __future__ import annotations

import hashlib
import json
import warnings
from dataclasses import dataclass
from pathlib import Path
from typing import Any

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.special import expit, logit
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.exceptions import ConvergenceWarning
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    average_precision_score,
    balanced_accuracy_score,
    brier_score_loss,
    confusion_matrix,
    precision_recall_curve,
    roc_auc_score,
    roc_curve,
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

from .model_data import stratified_mother_splits


Q4_FEATURES = (
    "z13",
    "z18",
    "z21",
    "x_zscore",
    "gc13",
    "gc18",
    "gc21",
    "gc_content",
    "raw_reads",
    "unique_mapped_reads",
    "mapping_ratio",
    "duplicate_ratio",
    "filtered_ratio",
    "bmi",
    "gestational_week",
    "age",
    "height_cm",
    "weight_kg",
)
MODEL_NAMES = ("z_rule", "elastic_net", "tree_ensemble")


@dataclass
class Q4Evaluation:
    oof_predictions: pd.DataFrame
    metrics: pd.DataFrame
    fold_audit: pd.DataFrame
    tuning: pd.DataFrame
    thresholds: pd.DataFrame
    selected_model: str
    prevalence: float


@dataclass
class CalibratedQ4Model:
    model_name: str
    features: tuple[str, ...]
```

```
    pipeline: Pipeline | None
    calibration_intercept: float
    calibration_slope: float
    threshold: float
    parameters: dict[str, Any]

    def _raw_probability(self, frame: pd.DataFrame) -> np.ndarray:
        if self.model_name == "z_rule":
            score = frame.loc[:, ["z13", "z18", "z21"]].abs().max(axis=1).to_numpy()
            return expit(2.0 * (score - 3.0))
        if self.pipeline is None:
            raise RuntimeError("Fitted Q4 pipeline is missing")
        return self.pipeline.predict_proba(frame.loc[:, self.features])[:, 1]

    def predict_probability(self, frame: pd.DataFrame) -> np.ndarray:
        raw = np.clip(self._raw_probability(frame), 1e-6, 1 - 1e-6)
        return expit(
            self.calibration_intercept + self.calibration_slope * logit(raw)
        )

    def predict(self, frame: pd.DataFrame) -> np.ndarray:
        return (self.predict_probability(frame) >= self.threshold).astype(int)


@dataclass
class Q4Analysis:
    data: pd.DataFrame
    evaluation: Q4Evaluation
    final_model: CalibratedQ4Model
    inconsistent_technical_events: int
    inconsistent_mothers: int
    calibration: pd.DataFrame
    confusion: pd.DataFrame
    feature_effects: pd.DataFrame
    sensitivity: pd.DataFrame
    bootstrap: pd.DataFrame


def prepare_q4_data(female: pd.DataFrame) -> pd.DataFrame:
    required = [*Q4_FEATURES, "mother_code", "technical_event_id", "aneuploidy_positive"]
    missing = [column for column in required if column not in female]
    if missing:
        raise ValueError(f"Q4 data are missing required fields: {missing}")
    data = female.copy(deep=True).reset_index(drop=True)
    data["target"] = data["aneuploidy_positive"].astype(int)
    data["row_index"] = np.arange(len(data), dtype=int)
    if data.loc[:, [*Q4_FEATURES, "mother_code", "technical_event_id", "target"]].isna().any().any():
        counts = data.loc[:, [*Q4_FEATURES, "mother_code", "technical_event_id", "target"]].isna().sum()
        raise ValueError(
            "Q4 data contain missing required values: "
            f"{counts.loc[counts.gt(0)].to_dict()}"
        )
    if data["target"].nunique() != 2:
        raise ValueError("Q4 target must contain both negative and positive AB labels")
    return data


def _candidate_parameters(model_name: str) -> list[dict[str, Any]]:
    if model_name == "z_rule":
        return [{"reference_z_threshold": 3.0}]
    if model_name == "elastic_net":
        return [
            {"C": c_value, "l1_ratio": l1_ratio}
            for c_value in (0.01, 0.05, 0.2)
            for l1_ratio in (0.2, 0.8)
        ]
    if model_name == "tree_ensemble":
        return [
            {
                "max_leaf_nodes": leaves,
                "min_samples_leaf": minimum_leaf,
                "l2_regularization": 5.0,
                "learning_rate": 0.05,
                "max_iter": 250,
            }
            for leaves in (7, 15)
            for minimum_leaf in (20, 40)
        ]
    raise ValueError(f"Unknown Q4 model: {model_name}")


def _build_pipeline(model_name: str, parameters: dict[str, Any], random_seed: int) -> Pipeline | None:
    if model_name == "z_rule":
        return None
    if model_name == "elastic_net":
        estimator = LogisticRegression(
            C=float(parameters["C"]),
            l1_ratio=float(parameters["l1_ratio"]),
            solver="saga",
            class_weight="balanced",
            max_iter=20_000,
            tol=1e-3,
            random_state=random_seed,
        )
        return Pipeline(
            [
                ("impute", SimpleImputer(strategy="median")),
                ("scale", StandardScaler()),
                ("model", estimator),
            ]
        )
    if model_name == "tree_ensemble":
        estimator = HistGradientBoostingClassifier(
            **parameters,
            class_weight="balanced",
            early_stopping=False,
            random_state=random_seed,
        )
        return Pipeline(
            [
```

```
                ("impute", SimpleImputer(strategy="median")),
                ("model", estimator),
            ]
        )
    raise ValueError(f"Unknown Q4 model: {model_name}")


def _raw_probability(
    model_name: str,
    pipeline: Pipeline | None,
    frame: pd.DataFrame,
) -> np.ndarray:
    if model_name == "z_rule":
        score = frame.loc[:, ["z13", "z18", "z21"]].abs().max(axis=1).to_numpy()
        return expit(2.0 * (score - 3.0))
    if pipeline is None:
        raise RuntimeError(f"Pipeline is missing for {model_name}")
    return pipeline.predict_proba(frame.loc[:, Q4_FEATURES])[:, 1]


def _fit_pipeline_checked(
    pipeline: Pipeline,
    frame: pd.DataFrame,
    target: np.ndarray,
) -> None:
    with warnings.catch_warnings(record=True) as caught:
        warnings.simplefilter("always")
        pipeline.fit(frame.loc[:, Q4_FEATURES], target)
    blocking = [
        item for item in caught if issubclass(item.category, ConvergenceWarning)
    ]
    if blocking:
        raise RuntimeError(
            "Q4 estimator emitted ConvergenceWarning; candidate is invalid"
        )


def _fit_platt(probabilities: np.ndarray, target: np.ndarray) -> tuple[float, float]:
    transformed = logit(np.clip(probabilities, 1e-6, 1 - 1e-6)).reshape(-1, 1)
    calibrator = LogisticRegression(C=1e6, solver="lbfgs", max_iter=5000)
    calibrator.fit(transformed, target)
    intercept = float(calibrator.intercept_[0])
    slope = float(calibrator.coef_[0, 0])
    if not np.isfinite(intercept) or not np.isfinite(slope) or slope <= 0:
        raise RuntimeError("Q4 Platt calibration did not produce a positive finite slope")
    return intercept, slope


def _apply_platt(probabilities: np.ndarray, intercept: float, slope: float) -> np.ndarray:
    return expit(intercept + slope * logit(np.clip(probabilities, 1e-6, 1 - 1e-6)))


def _inner_oof_raw_probability(
    data: pd.DataFrame,
    model_name: str,
    parameters: dict[str, Any],
    *,
    n_splits: int,
    random_seed: int,
) -> np.ndarray:
    target = data["target"].to_numpy(dtype=int)
    groups = data["mother_code"].astype(str).to_numpy()
    probabilities = np.full(len(data), np.nan, dtype=float)
    for fold, (train_idx, test_idx) in enumerate(
        stratified_mother_splits(
            target,
            groups,
            n_splits=n_splits,
            random_state=random_seed,
        )
    ):
        pipeline = _build_pipeline(model_name, parameters, random_seed + fold)
        if pipeline is not None:
            _fit_pipeline_checked(pipeline, data.iloc[train_idx], target[train_idx])
        probabilities[test_idx] = _raw_probability(
            model_name,
            pipeline,
            data.iloc[test_idx],
        )
    if np.isnan(probabilities).any():
        raise RuntimeError("Q4 inner grouped CV did not predict every row")
    return probabilities


def _tune_model(
    data: pd.DataFrame,
    model_name: str,
    *,
    inner_folds: int,
    random_seed: int,
) -> tuple[
    dict[str, Any],
    np.ndarray,
    np.ndarray,
    float,
    float,
    float,
    float,
    list[dict[str, Any]],
]:
    target = data["target"].to_numpy(dtype=int)
    best: tuple[dict[str, Any], np.ndarray, np.ndarray, float, float, float, float] | None = None
    best_pr_auc = -np.inf
    best_brier = np.inf
    invalid_candidates: list[dict[str, Any]] = []
    for candidate_index, parameters in enumerate(_candidate_parameters(model_name)):
        try:
            raw = _inner_oof_raw_probability(
                data,
                model_name,
```

```
                parameters,
                n_splits=inner_folds,
                random_seed=random_seed + 50 * candidate_index,
            )
        except RuntimeError as exc:
            invalid_candidates.append(
                {"parameters": parameters, "reason": str(exc)}
            )
            continue
        if model_name == "z_rule":
            intercept, slope = 0.0, 1.0
            calibrated = raw
        else:
            try:
                intercept, slope = _fit_platt(raw, target)
            except RuntimeError as exc:
                invalid_candidates.append(
                    {
                        "parameters": parameters,
                        "reason": str(exc),
                        "raw_roc_auc": float(roc_auc_score(target, raw)),
                    }
                )
                continue
            calibrated = _apply_platt(raw, intercept, slope)
        pr_auc = float(average_precision_score(target, calibrated))
        brier = float(brier_score_loss(target, calibrated))
        if pr_auc > best_pr_auc + 1e-12 or (
            abs(pr_auc - best_pr_auc) <= 1e-12 and brier < best_brier
        ):
            best = (parameters, raw, calibrated, pr_auc, brier, intercept, slope)
            best_pr_auc = pr_auc
            best_brier = brier
    if best is None:
        raise RuntimeError(
            f"Q4 tuning produced no valid candidate for {model_name}: {invalid_candidates}"
        )
    return (*best, invalid_candidates)


def _select_threshold(target: np.ndarray, probabilities: np.ndarray) -> tuple[float, float]:
    candidates = np.unique(np.concatenate(([0.0], probabilities, [1.0])))
    best_threshold = 0.5
    best_score = -np.inf
    best_sensitivity = -np.inf
    for threshold in candidates:
        predicted = (probabilities >= threshold).astype(int)
        score = float(balanced_accuracy_score(target, predicted))
        matrix = confusion_matrix(target, predicted, labels=[0, 1])
        sensitivity = float(matrix[1, 1] / matrix[1].sum()) if matrix[1].sum() else 0.0
        if score > best_score + 1e-12 or (
            abs(score - best_score) <= 1e-12
            and (sensitivity > best_sensitivity + 1e-12 or (
                abs(sensitivity - best_sensitivity) <= 1e-12
                and abs(float(threshold) - 0.5) < abs(best_threshold - 0.5)
            ))
        ):
            best_threshold = float(threshold)
            best_score = score
            best_sensitivity = sensitivity
    return best_threshold, best_score


def _expected_calibration_error(
    target: np.ndarray,
    probabilities: np.ndarray,
    bins: int = 10,
) -> float:
    error = 0.0
    for indices in np.array_split(np.argsort(probabilities), bins):
        if len(indices) == 0:
            continue
        error += len(indices) / len(target) * abs(
            float(np.mean(target[indices])) - float(np.mean(probabilities[indices]))
        )
    return float(error)


def _metric_row(
    model_name: str,
    target: np.ndarray,
    probabilities: np.ndarray,
    predicted: np.ndarray,
) -> dict[str, float | int | str]:
    tn, fp, fn, tp = confusion_matrix(target, predicted, labels=[0, 1]).ravel()
    return {
        "model": model_name,
        "pr_auc": float(average_precision_score(target, probabilities)),
        "roc_auc": float(roc_auc_score(target, probabilities)),
        "sensitivity": float(tp / (tp + fn)) if tp + fn else 0.0,
        "specificity": float(tn / (tn + fp)) if tn + fp else 0.0,
        "balanced_accuracy": float(balanced_accuracy_score(target, predicted)),
        "brier": float(brier_score_loss(target, probabilities)),
        "ece_10": _expected_calibration_error(target, probabilities, bins=10),
        "true_negative": int(tn),
        "false_positive": int(fp),
        "false_negative": int(fn),
        "true_positive": int(tp),
    }


def evaluate_q4_models(
    data: pd.DataFrame,
    *,
    outer_folds: int,
    inner_folds: int,
    random_seed: int,
) -> Q4Evaluation:
    target = data["target"].to_numpy(dtype=int)
    groups = data["mother_code"].astype(str).to_numpy()
```

```
predictions = {
    model_name: np.full(len(data), np.nan, dtype=float) for model_name in MODEL_NAMES
}
classifications = {
    model_name: np.full(len(data), -1, dtype=int) for model_name in MODEL_NAMES
}
outer_fold_column = np.full(len(data), -1, dtype=int)
audit_records: list[dict[str, int]] = []
tuning_records: list[dict[str, Any]] = []
threshold_records: list[dict[str, float | int | str]] = []
outer_splits = list(
    stratified_mother_splits(
        target,
        groups,
        n_splits=outer_folds,
        random_state=random_seed,
    )
)

for outer_fold, (train_idx, test_idx) in enumerate(outer_splits):
    outer_fold_column[test_idx] = outer_fold
    train_groups = set(groups[train_idx])
    test_groups = set(groups[test_idx])
    audit_records.append(
        {
            "outer_fold": outer_fold,
            "train_rows": len(train_idx),
            "test_rows": len(test_idx),
            "train_mothers": len(train_groups),
            "test_mothers": len(test_groups),
            "train_positive_rows": int(target[train_idx].sum()),
            "test_positive_rows": int(target[test_idx].sum()),
            "mother_overlap": len(train_groups.intersection(test_groups)),
        }
    )
    train_data = data.iloc[train_idx].reset_index(drop=True)
    train_target = target[train_idx]
    for model_position, model_name in enumerate(MODEL_NAMES):
        (
            parameters,
            _inner_raw,
            inner_probability,
            inner_pr_auc,
            inner_brier,
            calibration_intercept,
            calibration_slope,
            invalid_candidates,
        ) = _tune_model(
            train_data,
            model_name,
            inner_folds=inner_folds,
            random_seed=random_seed + 1000 * outer_fold + 100 * model_position,
        )
        if model_name == "z_rule":
            threshold = 0.5
            threshold_balanced_accuracy = float(
                balanced_accuracy_score(
                    train_target,
                    (inner_probability >= threshold).astype(int),
                )
            )
            threshold_source = "pre_specified_abs_z_3"
        else:
            threshold, threshold_balanced_accuracy = _select_threshold(
                train_target,
                inner_probability,
            )
            threshold_source = "inner_grouped_oof"
        pipeline = _build_pipeline(
            model_name,
            parameters,
            random_seed + 10_000 + outer_fold,
        )
        if pipeline is not None:
            _fit_pipeline_checked(pipeline, data.iloc[train_idx], train_target)
        raw_test = _raw_probability(model_name, pipeline, data.iloc[test_idx])
        calibrated_test = _apply_platt(
            raw_test,
            calibration_intercept,
            calibration_slope,
        )
        predictions[model_name][test_idx] = calibrated_test
        classifications[model_name][test_idx] = (calibrated_test >= threshold).astype(int)
        tuning_records.append(
            {
                "outer_fold": outer_fold,
                "model": model_name,
                "parameters": json.dumps(parameters, sort_keys=True),
                "inner_pr_auc": inner_pr_auc,
                "inner_brier": inner_brier,
                "calibration_intercept": calibration_intercept,
                "calibration_slope": calibration_slope,
                "invalid_candidates": json.dumps(invalid_candidates, sort_keys=True),
            }
        )
        threshold_records.append(
            {
                "outer_fold": outer_fold,
                "model": model_name,
                "threshold": threshold,
                "inner_balanced_accuracy": threshold_balanced_accuracy,
                "threshold_source": threshold_source,
            }
        )

if (outer_fold_column < 0).any():
    raise RuntimeError("Q4 outer grouped CV did not assign every row")
if any(np.isnan(values).any() for values in predictions.values()):
    raise RuntimeError("Q4 outer grouped CV did not predict every row")
if any((values < 0).any() for values in classifications.values()):
```

```
        raise RuntimeError("Q4 outer grouped CV did not classify every row")

    oof = pd.DataFrame(
        {
            "row_index": data["row_index"].to_numpy(dtype=int),
            "mother_code": groups,
            "technical_event_id": data["technical_event_id"].astype(str).to_numpy(),
            "target": target,
            "outer_fold": outer_fold_column,
            **{f"prob_{name}": values for name, values in predictions.items()},
            **{f"pred_{name}": values for name, values in classifications.items()},
        }
    )
    metrics = pd.DataFrame.from_records(
        [
            _metric_row(name, target, predictions[name], classifications[name])
            for name in MODEL_NAMES
        ]
    )
    candidate_metrics = metrics[metrics["model"].isin(["elastic_net", "tree_ensemble"])].copy()
    elastic_pr = float(candidate_metrics.loc[candidate_metrics["model"].eq("elastic_net"), "pr_auc"].iloc[0])
    tree_pr = float(candidate_metrics.loc[candidate_metrics["model"].eq("tree_ensemble"), "pr_auc"].iloc[0])
    selected_model = "elastic_net" if elastic_pr >= tree_pr - 0.01 else "tree_ensemble"
    return Q4Evaluation(
        oof_predictions=oof,
        metrics=metrics,
        fold_audit=pd.DataFrame.from_records(audit_records),
        tuning=pd.DataFrame.from_records(tuning_records),
        thresholds=pd.DataFrame.from_records(threshold_records),
        selected_model=selected_model,
        prevalence=float(np.mean(target)),
    )


def _fit_final_model(
    data: pd.DataFrame,
    model_name: str,
    *,
    random_seed: int,
) -> CalibratedQ4Model:
    (
        parameters,
        _raw,
        calibrated,
        _pr_auc,
        _brier,
        intercept,
        slope,
        _invalid_candidates,
    ) = _tune_model(
        data,
        model_name,
        inner_folds=5,
        random_seed=random_seed,
    )
    threshold = (
        0.5
        if model_name == "z_rule"
        else _select_threshold(data["target"].to_numpy(dtype=int), calibrated)[0]
    )
    pipeline = _build_pipeline(model_name, parameters, random_seed + 50_000)
    if pipeline is not None:
        _fit_pipeline_checked(pipeline, data, data["target"].to_numpy(dtype=int))
    return CalibratedQ4Model(
        model_name=model_name,
        features=Q4_FEATURES,
        pipeline=pipeline,
        calibration_intercept=intercept,
        calibration_slope=slope,
        threshold=threshold,
        parameters=parameters,
    )


def _calibration_table(evaluation: Q4Evaluation, bins: int = 10) -> pd.DataFrame:
    target = evaluation.oof_predictions["target"].to_numpy(dtype=int)
    records: list[dict[str, float | int | str]] = []
    for model_name in MODEL_NAMES:
        probabilities = evaluation.oof_predictions[f"prob_{model_name}"].to_numpy(dtype=float)
        for bin_index, indices in enumerate(np.array_split(np.argsort(probabilities), bins), start=1):
            if len(indices) == 0:
                continue
            records.append(
                {
                    "model": model_name,
                    "bin": bin_index,
                    "n": len(indices),
                    "mean_probability": float(np.mean(probabilities[indices])),
                    "observed_fraction": float(np.mean(target[indices])),
                }
            )
    return pd.DataFrame.from_records(records)


def _confusion_table(evaluation: Q4Evaluation) -> pd.DataFrame:
    row = evaluation.metrics.set_index("model").loc[evaluation.selected_model]
    return pd.DataFrame(
        [
            {"actual": 0, "predicted": 0, "count": int(row["true_negative"])},
            {"actual": 0, "predicted": 1, "count": int(row["false_positive"])},
            {"actual": 1, "predicted": 0, "count": int(row["false_negative"])},
            {"actual": 1, "predicted": 1, "count": int(row["true_positive"])},
        ]
    )


def _feature_effects(data: pd.DataFrame, model: CalibratedQ4Model) -> pd.DataFrame:
    if model.model_name == "z_rule":
        return pd.DataFrame(
            {
```

```
                "feature": ["z13", "z18", "z21"],
                "effect_type": "absolute_z_rule_component",
                "effect": [1.0, 1.0, 1.0],
                "absolute_effect": [1.0, 1.0, 1.0],
            }
        )
    if model.pipeline is None:
        raise RuntimeError("Q4 selected pipeline is missing")
    estimator = model.pipeline.named_steps["model"]
    if hasattr(estimator, "coef_"):
        coefficients = np.asarray(estimator.coef_, dtype=float).reshape(-1)
        return pd.DataFrame(
            {
                "feature": Q4_FEATURES,
                "effect_type": "standardized_log_odds_coefficient",
                "effect": coefficients,
                "absolute_effect": np.abs(coefficients),
            }
        ).sort_values("absolute_effect", ascending=False, kind="mergesort")

    frame = data.loc[:, Q4_FEATURES].copy()
    records: list[dict[str, float | str]] = []
    for feature in Q4_FEATURES:
        low = float(frame[feature].quantile(0.25))
        high = float(frame[feature].quantile(0.75))
        low_frame = frame.copy()
        high_frame = frame.copy()
        low_frame[feature] = low
        high_frame[feature] = high
        contrast = float(
            np.mean(model.predict_probability(high_frame))
            - np.mean(model.predict_probability(low_frame))
        )
        records.append(
            {
                "feature": feature,
                "effect_type": "mean_probability_iqr_contrast",
                "effect": contrast,
                "absolute_effect": abs(contrast),
            }
        )
    return pd.DataFrame.from_records(records).sort_values(
        "absolute_effect", ascending=False, kind="mergesort"
    )


def _sensitivity_table(data: pd.DataFrame, evaluation: Q4Evaluation) -> pd.DataFrame:
    selected = evaluation.selected_model
    base = evaluation.oof_predictions.copy()
    records: list[dict[str, float | int | str]] = []
    inconsistent_events = set(
        data.groupby("technical_event_id")["target"].nunique().loc[lambda value: value.gt(1)].index
    )
    scenarios = {
        "all_rows": np.ones(len(base), dtype=bool),
        "exclude_inconsistent_technical_events": ~base["technical_event_id"].isin(inconsistent_events).to_numpy(),
        "one_record_per_technical_event": ~base["technical_event_id"].duplicated(keep="first").to_numpy(),
    }
    for scenario, mask in scenarios.items():
        subset = base.loc[mask]
        metrics = _metric_row(
            selected,
            subset["target"].to_numpy(dtype=int),
            subset[f"prob_{selected}"].to_numpy(dtype=float),
            subset[f"pred_{selected}"].to_numpy(dtype=int),
        )
        records.append(
            {
                "scenario": scenario,
                "n_rows": len(subset),
                "positive_rows": int(subset["target"].sum()),
                **{key: value for key, value in metrics.items() if key != "model"},
            }
        )

    z_score = data.loc[:, ["z13", "z18", "z21"]].abs().max(axis=1).to_numpy(dtype=float)
    target = data["target"].to_numpy(dtype=int)
    z_probability = evaluation.oof_predictions["prob_z_rule"].to_numpy(dtype=float)
    for threshold in (2.5, 3.0, 3.5):
        metrics = _metric_row(
            "z_rule",
            target,
            z_probability,
            (z_score >= threshold).astype(int),
        )
        records.append(
            {
                "scenario": f"fixed_abs_z_threshold_{threshold:.1f}",
                "n_rows": len(data),
                "positive_rows": int(target.sum()),
                **{key: value for key, value in metrics.items() if key != "model"},
            }
        )
    return pd.DataFrame.from_records(records)


def _mother_bootstrap(
    evaluation: Q4Evaluation,
    *,
    replicates: int,
    random_seed: int,
) -> pd.DataFrame:
    selected = evaluation.selected_model
    oof = evaluation.oof_predictions
    mothers = oof["mother_code"].drop_duplicates().to_numpy()
    rng = np.random.default_rng(random_seed)
    records: list[dict[str, float | int | str]] = []
    groups = {mother: group for mother, group in oof.groupby("mother_code", sort=False)}
    for replicate in range(replicates):
        sampled = rng.choice(mothers, size=len(mothers), replace=True)
```

```
        resampled = pd.concat([groups[mother] for mother in sampled], ignore_index=True)
        target = resampled["target"].to_numpy(dtype=int)
        if np.unique(target).size < 2:
            records.append({"replicate": replicate, "status": "single_class"})
            continue
        metrics = _metric_row(
            selected,
            target,
            resampled[f"prob_{selected}"].to_numpy(dtype=float),
            resampled[f"pred_{selected}"].to_numpy(dtype=int),
        )
        records.append({"replicate": replicate, "status": "ok", **metrics})
    return pd.DataFrame.from_records(records)


def analyze_q4(
    female: pd.DataFrame,
    *,
    evaluation: Q4Evaluation | None = None,
    random_seed: int = 2025,
    bootstrap_replicates: int = 500,
) -> Q4Analysis:
    data = prepare_q4_data(female)
    if evaluation is None:
        evaluation = evaluate_q4_models(
            data,
            outer_folds=5,
            inner_folds=4,
            random_seed=random_seed,
        )
    final_model = _fit_final_model(
        data,
        evaluation.selected_model,
        random_seed=random_seed,
    )
    inconsistent_events = int(
        data.groupby("technical_event_id")["target"].nunique().gt(1).sum()
    )
    inconsistent_mothers = int(
        data.groupby("mother_code")["target"].nunique().gt(1).sum()
    )
    return Q4Analysis(
        data=data,
        evaluation=evaluation,
        final_model=final_model,
        inconsistent_technical_events=inconsistent_events,
        inconsistent_mothers=inconsistent_mothers,
        calibration=_calibration_table(evaluation),
        confusion=_confusion_table(evaluation),
        feature_effects=_feature_effects(data, final_model),
        sensitivity=_sensitivity_table(data, evaluation),
        bootstrap=_mother_bootstrap(
            evaluation,
            replicates=bootstrap_replicates,
            random_seed=random_seed,
        ),
    )


def _save_figure(fig: plt.Figure, output_dir: Path, stem: str) -> None:
    fig.savefig(output_dir / f"{stem}.pdf", bbox_inches="tight")
    fig.savefig(output_dir / f"{stem}.png", dpi=220, bbox_inches="tight")
    plt.close(fig)


def _plot_roc_pr(analysis: Q4Analysis, output_dir: Path) -> None:
    target = analysis.evaluation.oof_predictions["target"].to_numpy(dtype=int)
    fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.5))
    for model_name in MODEL_NAMES:
        probabilities = analysis.evaluation.oof_predictions[f"prob_{model_name}"].to_numpy(dtype=float)
        false_positive, true_positive, _ = roc_curve(target, probabilities)
        precision, recall, _ = precision_recall_curve(target, probabilities)
        axes[0].plot(false_positive, true_positive, label=model_name)
        axes[1].plot(recall, precision, label=model_name)
    axes[0].plot([0, 1], [0, 1], "--", color="0.6")
    axes[0].set(xlabel="False-positive rate", ylabel="Sensitivity", xlim=(0, 1), ylim=(0, 1))
    axes[1].axhline(analysis.evaluation.prevalence, linestyle="--", color="0.6")
    axes[1].set(xlabel="Recall", ylabel="Precision", xlim=(0, 1), ylim=(0, 1))
    for ax in axes:
        ax.legend(frameon=False, fontsize=8)
    fig.tight_layout()
    _save_figure(fig, output_dir, "q4_roc_pr")


def _plot_calibration(analysis: Q4Analysis, output_dir: Path) -> None:
    fig, ax = plt.subplots(figsize=(5.6, 5.0))
    ax.plot([0, 1], [0, 1], "--", color="0.55", label="Ideal")
    for model_name, group in analysis.calibration.groupby("model", sort=False):
        ax.plot(group["mean_probability"], group["observed_fraction"], marker="o", label=model_name)
    ax.set(xlabel="Predicted probability", ylabel="Observed fraction", xlim=(0, 1), ylim=(0, 1))
    ax.legend(frameon=False, fontsize=8)
    fig.tight_layout()
    _save_figure(fig, output_dir, "q4_calibration")


def _plot_confusion(analysis: Q4Analysis, output_dir: Path) -> None:
    matrix = analysis.confusion.pivot(index="actual", columns="predicted", values="count").to_numpy()
    fig, ax = plt.subplots(figsize=(4.8, 4.2))
    image = ax.imshow(matrix, cmap="Blues")
    for row in range(2):
        for column in range(2):
            ax.text(column, row, str(int(matrix[row, column])), ha="center", va="center")
    ax.set_xticks([0, 1], ["Pred 0", "Pred 1"])
    ax.set_yticks([0, 1], ["Actual 0", "Actual 1"])
    ax.set_title(analysis.evaluation.selected_model)
    fig.colorbar(image, ax=ax, fraction=0.046, pad=0.04)
    fig.tight_layout()
    _save_figure(fig, output_dir, "q4_confusion_matrix")
```

```
def _plot_feature_effects(analysis: Q4Analysis, output_dir: Path) -> None:
    shown = analysis.feature_effects.head(12).sort_values("absolute_effect", ascending=True)
    colors = np.where(shown["effect"].to_numpy(dtype=float) >= 0, "#E45756", "#4C78A8")
    fig, ax = plt.subplots(figsize=(7.2, 5.2))
    ax.barh(shown["feature"], shown["effect"], color=colors)
    ax.axvline(0, color="0.3", linewidth=0.8)
    ax.set_xlabel(str(shown["effect_type"].iloc[0]))
    fig.tight_layout()
    _save_figure(fig, output_dir, "q4_feature_effects")


def _sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def write_q4_outputs(analysis: Q4Analysis, output_dir: str | Path) -> dict[str, Any]:
    destination = Path(output_dir)
    destination.mkdir(parents=True, exist_ok=True)
    selected_metrics = analysis.evaluation.metrics.set_index("model").loc[
        analysis.evaluation.selected_model
    ]
    successful_bootstrap = analysis.bootstrap.loc[analysis.bootstrap["status"].eq("ok")]
    bootstrap_intervals = {
        metric: {
            "lower_95": float(successful_bootstrap[metric].quantile(0.025)),
            "upper_95": float(successful_bootstrap[metric].quantile(0.975)),
        }
        for metric in ("pr_auc", "roc_auc", "sensitivity", "specificity", "balanced_accuracy", "brier")
    }
    metrics_payload = {
        "schema_version": "1.0",
        "target": "AB_nonempty",
        "target_boundary": "Attachment AB detection label; AE is not a gold-standard outcome",
        "selected_model": analysis.evaluation.selected_model,
        "selected_parameters": analysis.final_model.parameters,
        "final_threshold": analysis.final_model.threshold,
        "prevalence": analysis.evaluation.prevalence,
        "rows": len(analysis.data),
        "mothers": int(analysis.data["mother_code"].nunique()),
        "positive_rows": int(analysis.data["target"].sum()),
        "inconsistent_technical_events": analysis.inconsistent_technical_events,
        "inconsistent_mothers": analysis.inconsistent_mothers,
        "outer_group_overlap_max": int(analysis.evaluation.fold_audit["mother_overlap"].max()),
        "all_outer_rows_predicted_once": bool(
            analysis.evaluation.oof_predictions["row_index"].nunique() == len(analysis.data)
        ),
        "selected_model_metrics": {
            key: float(value) for key, value in selected_metrics.items()
        },
        "mother_bootstrap_replicates": int(len(analysis.bootstrap)),
        "mother_bootstrap_failures": int((analysis.bootstrap["status"] != "ok").sum()),
        "mother_bootstrap_intervals": bootstrap_intervals,
    }
    (destination / "q4_metrics.json").write_text(
        json.dumps(metrics_payload, ensure_ascii=False, sort_keys=True, indent=2) + "\n",
        encoding="utf-8",
        newline="\n",
    )
    analysis.evaluation.oof_predictions.to_csv(destination / "q4_oof_predictions.csv", index=False, lineterminator="\n")
    analysis.evaluation.metrics.to_csv(destination / "q4_cv_metrics.csv", index=False, lineterminator="\n")
    analysis.evaluation.fold_audit.to_csv(destination / "q4_fold_audit.csv", index=False, lineterminator="\n")
    analysis.evaluation.tuning.to_csv(destination / "q4_tuning.csv", index=False, lineterminator="\n")
    analysis.evaluation.thresholds.to_csv(destination / "q4_thresholds.csv", index=False, lineterminator="\n")
    analysis.confusion.to_csv(destination / "q4_confusion_matrix.csv", index=False, lineterminator="\n")
    analysis.calibration.to_csv(destination / "q4_calibration.csv", index=False, lineterminator="\n")
    analysis.feature_effects.to_csv(destination / "q4_feature_effects.csv", index=False, lineterminator="\n")
    analysis.sensitivity.to_csv(destination / "q4_sensitivity.csv", index=False, lineterminator="\n")
    analysis.bootstrap.to_csv(destination / "q4_mother_bootstrap.csv", index=False, lineterminator="\n")

    _plot_roc_pr(analysis, destination)
    _plot_calibration(analysis, destination)
    _plot_confusion(analysis, destination)
    _plot_feature_effects(analysis, destination)

    artifacts: dict[str, dict[str, int | str]] = {}
    for path in sorted(destination.iterdir()):
        if path.is_file() and path.name != "q4_manifest.json":
            artifacts[path.name] = {"sha256": _sha256(path), "bytes": path.stat().st_size}
    manifest = {
        "schema_version": "1.0",
        "selected_model": analysis.evaluation.selected_model,
        "artifacts": artifacts,
    }
    (destination / "q4_manifest.json").write_text(
        json.dumps(manifest, ensure_ascii=False, sort_keys=True, indent=2) + "\n",
        encoding="utf-8",
        newline="\n",
    )
    return manifest
```

04_code/cumcm2025c/pipeline.py

```
from __future__ import annotations

import importlib.metadata
import json
import platform
import re
import sys
import traceback
from pathlib import Path
from typing import Any
```

```
import pandas as pd

from workflow_core.evidence.artifacts import sha256_file
from workflow_core.orchestration.run_context import RunContext

from .audit import write_processed_bundle
from .model_data import aggregate_male_technical_events
from .preprocessing import load_official_workbook
from .q1_longitudinal import fit_q1_models, prepare_q1_data, write_q1_outputs
from .q2_timing import (
    analyze_q2,
    bootstrap_q2_policy,
    simulate_measurement_error_policy,
    write_q2_outputs,
)
from .q3_multifactor import analyze_q3, write_q3_outputs
from .q4_classifier import analyze_q4, write_q4_outputs


FORMAL_STAGES = ("data", "q1", "q2", "q3", "q4", "evidence")
PIPELINE_RUN_KINDS = ("analysis", "reproduction")
REQUIRED_STAGE_OUTPUTS: dict[str, tuple[str, ...]] = {
    "data": (
        "data_audit.json",
        "female_clean.csv",
        "male_clean.csv",
        "male_events.csv",
        "manifest.json",
    ),
    "q1": (
        "q1_metrics.json",
        "q1_coefficients.csv",
        "q1_curves.csv",
        "q1_diagnostics.pdf",
        "q1_manifest.json",
    ),
    "q2": (
        "q2_metrics.json",
        "q2_group_policy.csv",
        "q2_measurement_error_simulation.csv",
        "q2_bootstrap.csv",
        "q2_sensitivity.pdf",
        "q2_manifest.json",
    ),
    "q3": (
        "q3_metrics.json",
        "q3_cv_metrics.csv",
        "q3_group_policy.csv",
        "q3_sensitivity.csv",
        "q3_calibration.pdf",
        "q3_manifest.json",
    ),
    "q4": (
        "q4_metrics.json",
        "q4_oof_predictions.csv",
        "q4_cv_metrics.csv",
        "q4_sensitivity.csv",
        "q4_calibration.pdf",
        "q4_manifest.json",
    ),
    "evidence": ("run_summary.json",),
}


def normalize_run_kind(value: str) -> str:
    run_kind = str(value).strip().lower()
    if run_kind not in PIPELINE_RUN_KINDS:
        raise ValueError("C pipeline run_kind must be analysis or reproduction")
    return run_kind


def validate_required_outputs(output_root: str | Path) -> list[Path]:
    root = Path(output_root)
    expected = [root / stage / name for stage, names in REQUIRED_STAGE_OUTPUTS.items() for name in names]
    missing = [path.relative_to(root).as_posix() for path in expected if not path.is_file()]
    if missing:
        raise FileNotFoundError("Formal output bundle is incomplete: " + ", ".join(missing))
    return expected


def _discover_inputs(project_root: Path) -> tuple[Path, Path]:
    inbox = project_root / "00_inbox"
    workbooks = sorted(inbox.glob("*.xlsx"))
    pdfs = sorted(inbox.glob("*.pdf"))
    if len(workbooks) != 1 or len(pdfs) != 1:
        raise RuntimeError(
            "Formal run requires exactly one XLSX and one PDF in 00_inbox; "
            f"found {len(workbooks)} XLSX and {len(pdfs)} PDF files"
        )
    return workbooks[0], pdfs[0]


def _configuration_paths(project_root: Path, repository_root: Path) -> list[str]:
    candidates = [
        project_root / "IMPLEMENTATION_PLAN.md",
        project_root / "WORKFLOW_ISSUES.md",
        project_root / "requirements.txt",
        project_root / "requirements.lock.txt",
        project_root / "01_problem/problem_analysis.md",
        project_root / "01_problem/problem_manifest.csv",
        project_root / "01_problem/checksums.sha256",
    ]
    candidates.extend(sorted((project_root / "config").rglob("*.yml")))
    candidates.extend(sorted((project_root / "04_code").rglob("*.py")))
    return [path.relative_to(repository_root).as_posix() for path in candidates if path.is_file()]


def _package_versions() -> dict[str, str]:
    names = (
```

```
        "numpy",
        "pandas",
        "scipy",
        "statsmodels",
        "scikit-learn",
        "matplotlib",
        "openpyxl",
        "pytest",
    )
    versions: dict[str, str] = {}
    for name in names:
        try:
            versions[name] = importlib.metadata.version(name)
        except importlib.metadata.PackageNotFoundError:
            versions[name] = "NOT_INSTALLED"
    return versions


def _read_json(path: Path) -> dict[str, Any]:
    payload = json.loads(path.read_text(encoding="utf-8"))
    if not isinstance(payload, dict):
        raise TypeError(f"Expected JSON object: {path}")
    return payload


def _write_run_summary(output_root: Path, *, random_seed: int) -> Path:
    q1 = _read_json(output_root / "q1/q1_metrics.json")
    q2 = _read_json(output_root / "q2/q2_metrics.json")
    q3 = _read_json(output_root / "q3/q3_metrics.json")
    q4 = _read_json(output_root / "q4/q4_metrics.json")
    summary = {
        "schema_version": "1.0",
        "random_seed": random_seed,
        "python": platform.python_version(),
        "executable": str(Path(sys.executable).resolve()),
        "packages": _package_versions(),
        "questions": {
            "Q1": {
                "selected_model": q1["selected_model"],
                "metrics": q1["metrics"],
                "robustness_tests": q1["robustness_tests"],
            },
            "Q2": {
                "selected_group_count": q2["selected_group_count"],
                "policy": q2["policy"],
                "bootstrap_failures": q2["bootstrap_failures"],
                "measurement_error_failures": q2["measurement_error_failures"],
            },
            "Q3": {
                "selected_model": q3["selected_model"],
                "selected_model_metrics": q3["selected_model_metrics"],
                "selected_group_count": q3["selected_group_count"],
            },
            "Q4": {
                "target": q4["target"],
                "selected_model": q4["selected_model"],
                "selected_model_metrics": q4["selected_model_metrics"],
                "inconsistent_technical_events": q4["inconsistent_technical_events"],
            },
        },
    }
    path = output_root / "evidence/run_summary.json"
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(
        json.dumps(summary, ensure_ascii=False, sort_keys=True, indent=2) + "\n",
        encoding="utf-8",
        newline="\n",
    )
    return path


def _media_type(path: Path) -> str:
    return {
        ".csv": "text/csv",
        ".json": "application/json",
        ".txt": "text/plain",
        ".pdf": "application/pdf",
        ".png": "image/png",
    }.get(path.suffix.lower(), "application/octet-stream")


def _artifact_id(relative_path: Path) -> str:
    safe = re.sub(r"[^A-Za-z0-9]+", "-", relative_path.as_posix()).strip("-")
    return f"C-FORMAL-{safe}"[:180]


def _register_outputs(context: RunContext) -> None:
    for path in sorted(item for item in (context.run_dir / "outputs").rglob("*") if item.is_file()):
        relative = path.relative_to(context.run_dir / "outputs")
        stage = relative.parts[0]
        model_id = {
            "q1": "Q1-MIXED-SPLINE",
            "q2": "Q2-GEE-DP",
            "q3": "Q3-GROUPED-CV",
            "q4": "Q4-NESTED-GROUP-CV",
        }.get(stage)
        context.register_artifact(
            artifact_id=_artifact_id(relative),
            stage=stage,
            producer=f"cumcm2025c.pipeline:{stage}",
            path=path,
            media_type=_media_type(path),
            source_artifacts=("RAW-C-ATTACHMENT",),
            model_id=model_id,
            distribution="internal",
        )


def run_formal_pipeline(
    project_root: str | Path,
```

```
    *,
    run_id: str,
    random_seed: int = 2025,
    q2_replicates: int = 200,
    q4_bootstrap_replicates: int = 500,
    run_kind: str = "analysis",
) -> Path:
    project = Path(project_root).resolve()
    repository = project.parents[1]
    workbook, problem_pdf = _discover_inputs(project)
    context: RunContext | None = None
    current_stage = "initialization"
    source_hash_before = sha256_file(workbook)
    resolved_run_kind = normalize_run_kind(run_kind)
    try:
        context = RunContext.create(
            project,
            run_kind=resolved_run_kind,
            profile="audit",
            run_id=run_id,
        )
        context.record_input_checksums(
            [
                {
                    "asset_id": "RAW-C-ATTACHMENT",
                    "logical_uri": "asset://raw/c-attachment",
                    "sha256": source_hash_before,
                    "size_bytes": workbook.stat().st_size,
                },
                {
                    "asset_id": "OFFICIAL-C-PROBLEM",
                    "logical_uri": "asset://official/c-problem",
                    "sha256": sha256_file(problem_pdf),
                    "size_bytes": problem_pdf.stat().st_size,
                },
            ]
        )
        context.record_configuration(
            repository,
            _configuration_paths(project, repository),
        )
        loaded = load_official_workbook(workbook)
        events = aggregate_male_technical_events(loaded.male)
        output_root = context.run_dir / "outputs"

        current_stage = "data"
        print("[data] exporting a fresh processed bundle", flush=True)
        context.update_stage(current_stage, "running")
        write_processed_bundle(loaded, output_root / current_stage)
        context.update_stage(
            current_stage,
            "completed",
            report_path="outputs/data/data_audit.json",
            details={"male_rows": len(loaded.male), "female_rows": len(loaded.female), "male_events": len(events)},
        )

        current_stage = "q1"
        print("[q1] fitting longitudinal mixed and robust models", flush=True)
        context.update_stage(current_stage, "running")
        q1_data = prepare_q1_data(events)
        q1_results = fit_q1_models(q1_data)
        write_q1_outputs(q1_data, q1_results, output_root / current_stage)
        context.update_stage(current_stage, "completed", report_path="outputs/q1/q1_metrics.json")

        current_stage = "q2"
        print(f"[q2] fitting timing policy with {q2_replicates}+{q2_replicates} mother-level resamples", flush=True)
        context.update_stage(current_stage, "running")
        q2_analysis = analyze_q2(loaded.male, events)
        q2_measurement = simulate_measurement_error_policy(
            loaded.male,
            events,
            n_replicates=q2_replicates,
            random_seed=random_seed + 1,
            n_groups=q2_analysis.policy.n_groups,
        )
        q2_bootstrap = bootstrap_q2_policy(
            events,
            n_replicates=q2_replicates,
            random_seed=random_seed + 2,
            n_groups=q2_analysis.policy.n_groups,
        )
        if not q2_measurement["status"].eq("ok").all() or not q2_bootstrap["status"].eq("ok").all():
            raise RuntimeError("Q2 formal resampling contains failed or non-converged fits")
        write_q2_outputs(q2_analysis, q2_measurement, q2_bootstrap, output_root / current_stage)
        context.update_stage(current_stage, "completed", report_path="outputs/q2/q2_metrics.json")

        current_stage = "q3"
        print("[q3] running nested mother-grouped multifactor validation", flush=True)
        context.update_stage(current_stage, "running")
        q3_analysis = analyze_q3(events, random_seed=random_seed)
        write_q3_outputs(q3_analysis, output_root / current_stage)
        context.update_stage(current_stage, "completed", report_path="outputs/q3/q3_metrics.json")

        current_stage = "q4"
        print("[q4] running nested mother-grouped female classification", flush=True)
        context.update_stage(current_stage, "running")
        q4_analysis = analyze_q4(
            loaded.female,
            random_seed=random_seed,
            bootstrap_replicates=q4_bootstrap_replicates,
        )
        write_q4_outputs(q4_analysis, output_root / current_stage)
        context.update_stage(current_stage, "completed", report_path="outputs/q4/q4_metrics.json")

        current_stage = "evidence"
        print("[evidence] validating required outputs and freezing provenance", flush=True)
        context.update_stage(current_stage, "running")
        _write_run_summary(output_root, random_seed=random_seed)
        validate_required_outputs(output_root)
        if sha256_file(workbook) != source_hash_before:
```

```
            raise RuntimeError("Official workbook changed during the formal run")
        context.update_stage(
            current_stage,
            "completed",
            report_path="outputs/evidence/run_summary.json",
            details={"source_sha256_unchanged": True},
        )
        _register_outputs(context)
        context.complete()
        if not context.verify_integrity(mark_tampered=True):
            raise RuntimeError("Completed C analysis run failed artifact integrity verification")
        context.freeze()
        print(f"[complete] frozen run {context.run_id}", flush=True)
        return context.run_dir
    except Exception:
        if context is not None and context.status not in {"failed", "frozen", "tampered"}:
            error_path = context.run_dir / "logs/pipeline_error.txt"
            error_path.write_text(traceback.format_exc(), encoding="utf-8", newline="\n")
            try:
                context.update_stage(
                    current_stage,
                    "failed",
                    report_path="logs/pipeline_error.txt",
                    details={"exception_type": sys.exc_info()[0].__name__ if sys.exc_info()[0] else "Unknown"},
                )
                context.set_status("failed")
            except Exception:
                pass
        raise
```

04_code/cumcm2025c/paper_assets.py

```
from __future__ import annotations

import json
import shutil
from pathlib import Path
from typing import Any, Iterable

import pandas as pd

from workflow_core.evidence.artifacts import sha256_file
from workflow_core.orchestration.run_context import RunContext


def latex_escape(value: object) -> str:
    text = str(value)
    replacements = {
        "\\": r"\textbackslash{}",
        "&": r"\&",
        "%": r"\%",
        "$": r"\$",
        "#": r"\#",
        "_": r"\_",
        "{": r"\{",
        "}": r"\}",
        "~": r"\textasciitilde{}",
        "^": r"\textasciicircum{}",
    }
    return "".join(replacements.get(character, character) for character in text)


def effect_type_label(value: object) -> str:
    """Convert internal effect identifiers to compact paper-facing labels."""
    labels = {
        "standardized_log_odds_coefficient": "标准化对数优势系数",
        "mean_probability_iqr_contrast": "四分位概率差",
        "absolute_z_rule_component": "Z 值规则分量",
    }
    text = str(value)
    return labels.get(text, text.replace("_", " "))


def _read_json(path: Path) -> dict[str, Any]:
    payload = json.loads(path.read_text(encoding="utf-8"))
    if not isinstance(payload, dict):
        raise TypeError(f"Expected JSON object: {path}")
    return payload


def _format_number(value: object, digits: int = 3) -> str:
    number = float(value)
    if number == 0:
        return "0"
    if abs(number) < 0.001 or abs(number) >= 10_000:
        return f"{number:.2e}"
    return f"{number:.{digits}f}".rstrip("0").rstrip(".")


def _write_table(
    path: Path,
    *,
    caption: str,
    label: str,
    headers: Iterable[str],
    rows: Iterable[Iterable[object]],
    alignment: str,
) -> None:
    header_line = " & ".join(latex_escape(value) for value in headers) + r" \\"
    row_lines = [
        " & ".join(latex_escape(value) for value in row) + r" \\"
        for row in rows
    ]
    body = "\n".join(row_lines)
    content = (
        "\\begin{table}[H]\n"
```

```
        "\\centering\n"
        f"\\caption{{{caption}}}\n"
        f"\\label{{{label}}}\n"
        f"\\begin{{tabular}}{{{alignment}}}\n"
        "\\toprule\n"
        f"{header_line}\n"
        "\\midrule\n"
        f"{body}\n"
        "\\bottomrule\n"
        "\\end{tabular}\n"
        "\\end{table}\n"
    )
    path.write_text(content, encoding="utf-8", newline="\n")


def _macro(name: str, value: object) -> str:
    return f"\\newcommand{{\\{name}}}{{{latex_escape(value)}}}"


def _write_macros(
    path: Path,
    *,
    run_id: str,
    audit: dict[str, Any],
    q1: dict[str, Any],
    q2: dict[str, Any],
    q3: dict[str, Any],
    q4: dict[str, Any],
) -> None:
    q1_metrics = q1["metrics"]
    q2_policy = q2["policy"]
    q3_metrics = q3["selected_model_metrics"]
    q4_metrics = q4["selected_model_metrics"]
    values = {
        "AnalysisRunId": run_id,
        "MaleRows": int(audit["male"]["rows"]),
        "MaleMothers": int(audit["male"]["mothers"]),
        "MaleEvents": int(audit["male"]["technical_events"]),
        "FemaleRows": int(audit["female"]["rows"]),
        "FemaleMothers": int(audit["female"]["mothers"]),
        "FemalePositiveRows": int(audit["female"]["positive_aneuploidy_rows"]),
        "QOneMarginalRSquared": _format_number(q1_metrics["marginal_r2"]),
        "QOneConditionalRSquared": _format_number(q1_metrics["conditional_r2"]),
        "QOneICC": _format_number(q1_metrics["intraclass_correlation"]),
        "QOneWeekP": _format_number(q1["robustness_tests"]["gestational_week"]["p_value"]),
        "QOneBMIP": _format_number(q1["robustness_tests"]["bmi"]["p_value"]),
        "QTwoGroupCount": int(q2["selected_group_count"]),
        "QTwoMeanRegret": _format_number(q2_policy["mean_regret_weeks"]),
        "QTwoMeasurementSigma": _format_number(q2["measurement_error"]["measurement_sigma"], 4),
        "QTwoBootstrapFailures": int(q2["bootstrap_failures"]),
        "QThreeSelectedModel": q3["selected_model"],
        "QThreeROCAUC": _format_number(q3_metrics["roc_auc"]),
        "QThreePRAUC": _format_number(q3_metrics["pr_auc"]),
        "QThreeBrier": _format_number(q3_metrics["brier"]),
        "QThreeGroupCount": int(q3["selected_group_count"]),
        "QThreeMeanRegret": _format_number(q3["mean_policy_regret_weeks"]),
        "QFourSelectedModel": q4["selected_model"],
        "QFourPrevalence": _format_number(q4["prevalence"]),
        "QFourPRAUC": _format_number(q4_metrics["pr_auc"]),
        "QFourROCAUC": _format_number(q4_metrics["roc_auc"]),
        "QFourSensitivity": _format_number(q4_metrics["sensitivity"]),
        "QFourSpecificity": _format_number(q4_metrics["specificity"]),
        "QFourBalancedAccuracy": _format_number(q4_metrics["balanced_accuracy"]),
        "QFourBrier": _format_number(q4_metrics["brier"]),
        "QFourInconsistentEvents": int(q4["inconsistent_technical_events"]),
        "QFourInconsistentMothers": int(q4["inconsistent_mothers"]),
    }
    path.write_text(
        "% Generated from the frozen analysis run; do not edit numeric values manually.\n"
        + "\n".join(_macro(name, value) for name, value in values.items())
        + "\n",
        encoding="utf-8",
        newline="\n",
    )


def _copy_figures(run_dir: Path, figure_dir: Path, run_id: str) -> pd.DataFrame:
    mapping = {
        "Q1-RELATIONSHIP": ("q1/q1_relationship.pdf", "孕周与 BMI 条件效应曲线"),
        "Q1-DIAGNOSTICS": ("q1/q1_diagnostics.pdf", "问题一残差与影响诊断"),
        "Q2-PROBABILITY": ("q2/q2_probability_surface.pdf", "问题二达标概率曲面"),
        "Q2-POLICY": ("q2/q2_policy.pdf", "问题二 BMI 分组时点"),
        "Q2-SENSITIVITY": ("q2/q2_sensitivity.pdf", "问题二阈值与可靠性敏感性"),
        "Q3-CV": ("q3/q3_cv_comparison.pdf", "问题三组外模型比较"),
        "Q3-CALIBRATION": ("q3/q3_calibration.pdf", "问题三校准曲线"),
        "Q3-REGRET": ("q3/q3_policy_regret.pdf", "问题三策略压缩后悔值"),
        "Q4-ROC-PR": ("q4/q4_roc_pr.pdf", "问题四 ROC 与 PR 曲线"),
        "Q4-CALIBRATION": ("q4/q4_calibration.pdf", "问题四校准曲线"),
        "Q4-CONFUSION": ("q4/q4_confusion_matrix.pdf", "问题四混淆矩阵"),
        "Q4-EFFECTS": ("q4/q4_feature_effects.pdf", "问题四特征效应"),
    }
    records: list[dict[str, object]] = []
    for figure_id, (relative_source, purpose) in mapping.items():
        source = run_dir / "outputs" / relative_source
        if not source.is_file():
            raise FileNotFoundError(f"Frozen run figure is missing: {source}")
        destination = figure_dir / source.name
        shutil.copy2(source, destination)
        records.append(
            {
                "figure_id": figure_id,
                "file_name": destination.name,
                "source_run_id": run_id,
                "source_relative_path": source.relative_to(run_dir).as_posix(),
                "sha256": sha256_file(destination),
                "purpose": purpose,
            }
        )
    return pd.DataFrame.from_records(records)
```

```
def _write_tables(run_dir: Path, table_dir: Path, audit: dict[str, Any]) -> None:
    q1_comparison = pd.read_csv(run_dir / "outputs/q1/q1_model_comparison.csv")
    q2_policy = pd.read_csv(run_dir / "outputs/q2/q2_group_policy.csv")
    q3_metrics = pd.read_csv(run_dir / "outputs/q3/q3_cv_metrics.csv")
    q3_policy = pd.read_csv(run_dir / "outputs/q3/q3_group_policy.csv")
    q4_metrics = pd.read_csv(run_dir / "outputs/q4/q4_cv_metrics.csv")
    q4_confusion = pd.read_csv(run_dir / "outputs/q4/q4_confusion_matrix.csv")
    q4_effects = pd.read_csv(run_dir / "outputs/q4/q4_feature_effects.csv").head(8)

    _write_table(
        table_dir / "data_overview.tex",
        caption="附件数据结构与技术重复审计",
        label="tab:data-overview",
        headers=("数据集", "记录数", "孕妇数", "技术事件", "重复事件", "AB 阳性行"),
        rows=(
            ("男胎", audit["male"]["rows"], audit["male"]["mothers"], audit["male"]["technical_events"], audit["male"]["technical_repeat_events"], audit["male"]["positive_aneuploidy_rows"]),
            ("女胎", audit["female"]["rows"], audit["female"]["mothers"], audit["female"]["technical_events"], audit["female"]["technical_repeat_events"], audit["female"]["positive_aneuploidy_rows"]),
        ),
        alignment="lrrrrr",
    )
    _write_table(
        table_dir / "q1_model_comparison.tex",
        caption="问题一纵向候选模型比较",
        label="tab:q1-comparison",
        headers=("模型", "收敛", "AIC", "BIC", "固定效应参数数"),
        rows=(
            (row.model, bool(row.converged), _format_number(row.aic), _format_number(row.bic), int(row.fixed_effect_parameters))
            for row in q1_comparison.itertuples(index=False)
        ),
        alignment="lrrrr",
    )
    _write_table(
        table_dir / "q2_policy.tex",
        caption="问题二 BMI 连续分组与推荐检测时点",
        label="tab:q2-policy",
        headers=("组别", "BMI 下界", "BMI 上界", "孕妇数", "推荐孕周", "平均额外等待/周"),
        rows=(
            (int(row.group), _format_number(row.bmi_lower, 2), _format_number(row.bmi_upper, 2), int(row.n_mothers), _format_number(row.recommended_week, 1), _format_number(row.mean_excess_wait_weeks))
            for row in q2_policy.itertuples(index=False)
        ),
        alignment="crrrrr",
    )
    _write_table(
        table_dir / "q3_metrics.tex",
        caption="问题三按孕妇分组的组外预测性能",
        label="tab:q3-metrics",
        headers=("模型", "ROC-AUC", "PR-AUC", "Brier", "ECE"),
        rows=(
            (row.model, _format_number(row.roc_auc), _format_number(row.pr_auc), _format_number(row.brier), _format_number(row.ece_10))
            for row in q3_metrics.itertuples(index=False)
        ),
        alignment="lrrrr",
    )
    _write_table(
        table_dir / "q3_policy.tex",
        caption="问题三多因素策略压缩后的 BMI 分组",
        label="tab:q3-policy",
        headers=("组别", "BMI 下界", "BMI 上界", "孕妇数", "推荐孕周", "平均后悔/周"),
        rows=(
            (int(row.group), _format_number(row.bmi_lower, 2), _format_number(row.bmi_upper, 2), int(row.n_mothers), _format_number(row.recommended_week, 1), _format_number(row.mean_excess_wait_weeks))
            for row in q3_policy.itertuples(index=False)
        ),
        alignment="crrrrr",
    )
    _write_table(
        table_dir / "q4_metrics.tex",
        caption="问题四嵌套孕妇分组交叉验证性能",
        label="tab:q4-metrics",
        headers=("模型", "PR-AUC", "ROC-AUC", "灵敏度", "特异度", "平衡准确率", "Brier"),
        rows=(
            (row.model, _format_number(row.pr_auc), _format_number(row.roc_auc), _format_number(row.sensitivity), _format_number(row.specificity), _format_number(row.balanced_accuracy), _format_number(row.brier))
            for row in q4_metrics.itertuples(index=False)
        ),
        alignment="lrrrrrr",
    )
    _write_table(
        table_dir / "q4_confusion.tex",
        caption="问题四所选模型的组外混淆矩阵明细",
        label="tab:q4-confusion",
        headers=("真实标签", "预测标签", "记录数"),
        rows=((int(row.actual), int(row.predicted), int(row.count)) for row in q4_confusion.itertuples(index=False)),
        alignment="ccc",
    )
    _write_table(
        table_dir / "q4_effects.tex",
        caption="问题四所选模型的主要特征效应",
        label="tab:q4-effects",
        headers=("特征", "效应类型", "效应值"),
        rows=((row.feature, effect_type_label(row.effect_type), _format_number(row.effect)) for row in q4_effects.itertuples(index=False)),
        alignment="llr",
    )


def _write_evidence_tables(project: Path, run_id: str) -> None:
    evidence = project / "07_paper/evidence"
    claims = pd.DataFrame.from_records(
        [
            {"claim_id": "C-Q1-RELATION", "question_id": "Q1", "claim": "孕周为稳定显著因素，BMI 方向为负", "source": "q1/q1_metrics.json"},
            {"claim_id": "C-Q2-POLICY", "question_id": "Q2", "claim": "连续 BMI 分组与推荐周来自同一受约束目标", "source": "q2/q2_group_policy.csv"},
            {"claim_id": "C-Q3-GENERALIZATION", "question_id": "Q3", "claim": "多因素增益仅由未见孕妇 OOF 结果支持", "source": "q3/q3_cv_metrics.csv"},
            {"claim_id": "C-Q4-BOUNDARY", "question_id": "Q4", "claim": "性能针对 AB 检测标签而非 AE 出生结局", "source": "q4/q4_metrics.json"},
        ]
    )
    claims["run_id"] = run_id
    claims.to_csv(evidence / "claims.csv", index=False, lineterminator="\n")
    links = claims.loc[:, ["claim_id", "run_id", "source"]].copy()
    links["source_relative_path"] = "05_model_results/runs/" + run_id + "/outputs/" + links["source"]
    links.to_csv(evidence / "evidence_links.csv", index=False, lineterminator="\n")
    claims.loc[:, ["question_id", "claim_id", "claim"]].to_csv(
```

```
        evidence / "question_result_cards.csv", index=False, lineterminator="\n"
    )
    claims.loc[:, ["question_id", "claim"]].to_csv(
        evidence / "abstract_matrix.csv", index=False, lineterminator="\n"
    )


def generate_paper_assets(project_root: str | Path, run_id: str) -> dict[str, Any]:
    project = Path(project_root).resolve()
    context = RunContext.load(project, run_id)
    if context.status != "frozen":
        raise RuntimeError("Paper assets require a frozen analysis run")
    if not context.verify_integrity(mark_tampered=False):
        raise RuntimeError("Frozen analysis run failed integrity verification")

    run_dir = context.run_dir
    asset_root = project / "06_paper_assets"
    figure_dir = asset_root / "figures"
    table_dir = asset_root / "tables"
    metadata_dir = asset_root / "figure_metadata"
    for directory in (figure_dir, table_dir, metadata_dir):
        directory.mkdir(parents=True, exist_ok=True)

    audit = _read_json(run_dir / "outputs/data/data_audit.json")
    q1 = _read_json(run_dir / "outputs/q1/q1_metrics.json")
    q2 = _read_json(run_dir / "outputs/q2/q2_metrics.json")
    q3 = _read_json(run_dir / "outputs/q3/q3_metrics.json")
    q4 = _read_json(run_dir / "outputs/q4/q4_metrics.json")

    figure_manifest = _copy_figures(run_dir, figure_dir, run_id)
    figure_manifest.to_csv(asset_root / "figure_manifest.csv", index=False, lineterminator="\n")
    figure_manifest.to_csv(metadata_dir / "figure_sources.csv", index=False, lineterminator="\n")
    _write_tables(run_dir, table_dir, audit)
    _write_macros(
        table_dir / "results_macros.tex",
        run_id=run_id,
        audit=audit,
        q1=q1,
        q2=q2,
        q3=q3,
        q4=q4,
    )
    _write_evidence_tables(project, run_id)

    generated_files = sorted(
        path for path in asset_root.rglob("*") if path.is_file() and path.name != "asset_manifest.json"
    )
    manifest = {
        "schema_version": "1.0",
        "source_run_id": run_id,
        "source_run_integrity_verified": True,
        "artifacts": [
            {
                "relative_path": path.relative_to(project).as_posix(),
                "sha256": sha256_file(path),
                "size_bytes": path.stat().st_size,
            }
            for path in generated_files
        ],
    }
    (asset_root / "asset_manifest.json").write_text(
        json.dumps(manifest, ensure_ascii=False, sort_keys=True, indent=2) + "\n",
        encoding="utf-8",
        newline="\n",
    )
    if not context.verify_integrity(mark_tampered=False):
        raise RuntimeError("Source run changed while generating paper assets")
    return manifest
```

04_code/cumcm2025c/reproduction_compare.py

```
from __future__ import annotations

import json
from pathlib import Path
from typing import Any

from workflow_core.evidence.artifacts import sha256_file
from workflow_core.orchestration.run_context import RunContext


def _files(root: Path) -> dict[str, Path]:
    return {
        path.relative_to(root).as_posix(): path
        for path in sorted(root.rglob("*"))
        if path.is_file()
    }


def _normalized_artifact_manifest(path: Path) -> str:
    payload = json.loads(path.read_text(encoding="utf-8"))
    artifacts = payload.get("artifacts") if isinstance(payload, dict) else None
    if isinstance(artifacts, dict):
        for name, record in artifacts.items():
            if str(name).lower().endswith(".pdf") and isinstance(record, dict):
                record["bytes"] = "PDF_CONTAINER_METADATA"
                record["sha256"] = "PDF_CONTAINER_METADATA"
    return json.dumps(payload, ensure_ascii=False, sort_keys=True, separators=(",", ":"))


def compare_output_directories(reference_root: str | Path, candidate_root: str | Path) -> dict[str, Any]:
    reference = Path(reference_root).resolve()
    candidate = Path(candidate_root).resolve()
    reference_files = _files(reference)
    candidate_files = _files(candidate)
    paths = sorted(set(reference_files) | set(candidate_files))
```

```
    items: list[dict[str, Any]] = []
    counts = {
        "exact_match": 0,
        "pdf_container_only": 0,
        "normalized_manifest": 0,
        "mismatch": 0,
    }

    for relative in paths:
        left = reference_files.get(relative)
        right = candidate_files.get(relative)
        if left is None or right is None:
            status = "missing_from_candidate" if right is None else "extra_in_candidate"
            counts["mismatch"] += 1
        elif sha256_file(left) == sha256_file(right):
            status = "exact_match"
            counts[status] += 1
        elif relative.lower().endswith(".pdf"):
            png_relative = str(Path(relative).with_suffix(".png")).replace("\\", "/")
            left_png = reference_files.get(png_relative)
            right_png = candidate_files.get(png_relative)
            if left_png and right_png and sha256_file(left_png) == sha256_file(right_png):
                status = "pdf_container_only"
                counts[status] += 1
            else:
                status = "content_mismatch"
                counts["mismatch"] += 1
        elif relative.lower().endswith("_manifest.json"):
            if _normalized_artifact_manifest(left) == _normalized_artifact_manifest(right):
                status = "normalized_manifest"
                counts[status] += 1
            else:
                status = "content_mismatch"
                counts["mismatch"] += 1
        else:
            status = "content_mismatch"
            counts["mismatch"] += 1
        items.append({"relative_path": relative, "status": status})

    return {
        "schema_version": 1,
        "status": "PASSED" if counts["mismatch"] == 0 else "FAILED",
        "reference_file_count": len(reference_files),
        "candidate_file_count": len(candidate_files),
        "exact_match_count": counts["exact_match"],
        "pdf_container_only_count": counts["pdf_container_only"],
        "normalized_manifest_count": counts["normalized_manifest"],
        "mismatch_count": counts["mismatch"],
        "items": items,
    }


def compare_frozen_runs(project_root: str | Path, reference_run_id: str, candidate_run_id: str) -> dict[str, Any]:
    project = Path(project_root).resolve()
    reference = RunContext.load(project, reference_run_id)
    candidate = RunContext.load(project, candidate_run_id)
    lifecycle_ok = (
        reference.status == "frozen"
        and candidate.status == "frozen"
        and reference.manifest.get("run_kind") == "analysis"
        and candidate.manifest.get("run_kind") == "reproduction"
        and reference.verify_integrity(mark_tampered=False)
        and candidate.verify_integrity(mark_tampered=False)
    )
    reference_inputs = json.loads((reference.run_dir / "input_checksums.json").read_text(encoding="utf-8"))
    candidate_inputs = json.loads((candidate.run_dir / "input_checksums.json").read_text(encoding="utf-8"))
    inputs_equal = reference_inputs == candidate_inputs
    report = compare_output_directories(reference.run_dir / "outputs", candidate.run_dir / "outputs")
    report.update(
        {
            "reference_run_id": reference_run_id,
            "candidate_run_id": candidate_run_id,
            "reference_run_kind": reference.manifest.get("run_kind"),
            "candidate_run_kind": candidate.manifest.get("run_kind"),
            "lifecycle_and_integrity_ok": lifecycle_ok,
            "input_checksums_equal": inputs_equal,
            "pdf_comparison_note": "PDF byte differences are accepted only when the same-stem generated PNG is byte-identical; manifest normalization removes only PDF bytes and SHA256.",
        }
    )
    if not lifecycle_ok or not inputs_equal:
        report["status"] = "FAILED"
    return report
```